



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Laurea Magistrale in Informatica

Diffusion Echo State Networks for Text Generation

Relatori:

Prof. Claudio Gallicchio

Prof. Andrea Ceni

Dott. Giacomo Lagomarsini

Candidato:

Gabriele Benedetti

ANNO ACCADEMICO 2026/2027

Table of Contents

1	Introduction	4
1.1	Thesis Contributions	5
1.2	Thesis Structure	7
2	Background	8
2.1	Recurrent Neural Networks	8
2.2	Reservoir Computing and ParalESN	9
2.2.1	Echo State Networks (ESNs)	9
2.2.2	Parallel Echo State Networks (ParalESN)	10
2.3	Gated Recurrent Unit (GRU)	14
2.4	The Transformer Architecture	15
2.5	State-Space Models (SSM)	17
2.5.1	Discretization	17
2.5.2	S5	18
2.5.3	Linear Recurrent Units (LRU)	18
2.5.4	MAMBA	19
2.6	Diffusion Models	19
2.6.1	Continuous Diffusion Modeling	27
2.6.2	Masked Diffusion Language Models	30
2.6.3	Soft-Masked Diffusion Models	34
3	Diffusion Language Models in the Reservoir World	37
3.1	Model Architecture	37
3.2	Input and Conditioning	40
3.3	The DiESN Block	41
3.3.1	Mixing Stage	41
3.3.2	Feed-Forward Stage	43
3.4	Output Head	43
3.5	Trainable and Frozen Parameters	44
3.6	Training Objective	44
4	Experiments	45
4.1	ParalESN Configurations	45

4.2	Evaluation	47
4.3	Synthetic Datasets	48
4.3.1	TinyStories	48
4.3.2	TextBooks	50
4.4	Real-World Datasets	51
4.4.1	Wikipedia	51
4.4.2	Preliminary Transfer Experiment on OpenWebText	52
4.4.3	Observed Scaling Challenges in Text Diffusion	54
4.5	Targeted Benchmarks	55
4.5.1	Sequence Copy	55
4.5.2	Dyck-4 Language Test	58
4.5.3	Computational Efficiency on Long Contexts	60
4.5.4	VRAM Usage	61
4.5.5	Inference Comparison	62
4.5.6	DiESN vs. Autoregressive (Qwen1.5-0.5B)	63
4.5.7	Ablation	64
4.6	ParalESN vs. S5: The Impact of Accelerated Scan	65
5	Conclusions	69
	Bibliography	71

Chapter 1

Introduction

Processing sequential data and generating coherent natural language are among the most critical challenges in modern Artificial Intelligence. Generative Large Language Models (LLMs) have revolutionized natural language processing, largely driven by autoregressive (AR) Transformer architectures [1]. Despite their remarkable success, AR models suffer from a fundamental limitation: they generate tokens sequentially, one after the other. This sequential nature makes inference computationally expensive and leads to high latency. Furthermore, the core component of Transformers, the self-attention mechanism, scales quadratically with respect to the sequence length and this quadratic complexity poses a severe bottleneck when models have to process large contexts and generate long sequences.

Originally developed for continuous domains like image and audio generation, recently diffusion models [2] have emerged as a highly promising alternative for language modeling. These Diffusion Language Models [3] (DLMs) offer distinct advantages over traditional autoregressive (AR) approaches, including bidirectional context modeling, parallel and accelerated sampling, and built-in self-correction mechanisms. Masked Diffusion Language Models [4] (MDLMs), in particular, operate in discrete text spaces by progressively corrupting text into a sequence of mask tokens and then training a network to iteratively denoise them. Advanced techniques, such as soft-masking [5], further enhance these models by replacing binary masking decisions with a superposition of top predictions, allowing the model to propagate richer continuous feedback across decoding steps.

However, state-of-the-art MDLMs still rely on Transformers as their denoising backbone. Consequently, they inherit the quadratic time and memory complexity of self-attention, which hinders their efficiency when

dealing with long sequences. To overcome this quadratic bottleneck, Recurrent Neural Networks (RNNs) [6] represent a classical alternative, offering linear time complexity with respect to sequence length. However, their inherently sequential processing prevents parallelization during training, leading to prohibitive training times on large datasets. Furthermore, standard RNNs suffer from the vanishing and exploding gradient problems [7], [8] when trained via Backpropagation Through Time (BPTT) [9], which limits their ability to capture long-range dependencies effectively.

To address these limitations, this thesis explores the integration of Reservoir Computing (RC) [10], [11] principles into the diffusion framework. Reservoir Computing is a paradigm that leverages fixed, untrained recurrent networks (the Reservoir) [12] to process temporal data, requiring only a simple linear readout to be trained. This approach offers exceptional training efficiency by completely bypassing backpropagation through time (BPTT) [9]. While traditional RC models process data sequentially and struggle with the memory footprint of dense high-dimensional reservoirs, recent advancements have bridged the gap between RC and modern structured state spaces. Specifically, the Parallel Echo State Network (ParalESN) [13] employs a diagonal linear recurrence in the complex domain. This structure enables the parallel processing of temporal data via associative scans [14], reducing the computational complexity from quadratic to linear with respect to the sequence length, while maintaining the expressive power of the network.

1.1 Thesis Contributions

The primary objective of this thesis is to design and evaluate a novel diffusion model for text generation capable of handling long sequences in linear time, without compromising the quality of the generated text. We also aim to demonstrate that ParalESN is a highly competitive alternative to Transformers across various scenarios.

To achieve this, we propose an architecture where the global context management—traditionally handled by the query, key, and value projections of self-attention—is replaced by ParalESN. Because the recurrent reservoir in ParalESN is initialized and kept frozen, the sequence-mixing component requires no backpropagation. This design should reduce the trainable para-

meter count and memory footprint, bringing performance improvements and efficiency benefits.

Specifically, our contributions are as follows:

- We introduce a novel diffusion backbone that integrates bidirectional ParalESN blocks, Adaptive Layer Normalization (AdaLN) conditioning, and local depthwise convolutions.
- We adapt the soft-masking mechanism to work seamlessly with our linear recurrent backbone, allowing the model to leverage continuous predictive feedback during the iterative decoding process.
- We conduct extensive experiments to benchmark the proposed model against baseline Transformer-based diffusion architectures, specifically aiming to demonstrate that:
 - On **short sequences**, ParalESN maintains competitive results with respect to Transformer-based architectures.
 - On **long sequences**, ParalESN significantly outperforms the Transformer in terms of computational time, while keeping comparable perplexity (PPL).
 - In tasks requiring **memory**, ParalESN achieves optimal results by storing past information, despite not being a fully trained model.

Overall, we show that a non-trainable, linear recurrent reservoir can effectively guide the complex denoising process of discrete diffusion models, performing as well as—or even better than—fully trainable models. We also discuss the main findings, the limitations encountered during the development, and indicate potential future developments for this line of research.

The core contributions of this thesis have been accepted for presentation at the NeurIPS 2026 Workshop (AXIOM) [15].

1.2 Thesis Structure

The remainder of this thesis is organized as follows:

- **Chapter 2** provides the necessary theoretical background. It introduces sequence modeling concepts, focusing on Diffusion Language Models, discrete masking, and soft-masking techniques. It also covers the fundamentals of Reservoir Computing, Echo State Networks, and State Space Models.
- **Chapter 3** details the proposed architecture. We thoroughly describe the integration of ParalESN into the diffusion framework, explaining the design of the bidirectional recurrent blocks, the AdaLN conditioning, and the embedding layers.
- **Chapter 4** presents the experimental setup and the results. We benchmark the proposed model on selected datasets, analyzing its generative performance and comparing its computational efficiency (training and inference times, memory usage) against other state-of-the-art models.
- **Chapter 5** concludes the work, summarizing the main findings, discussing the limitations of the proposed approach, and outlining potential directions for future research.

Chapter 2

Background

This chapter outlines the theoretical foundations underpinning the development of the Soft Masked Diffusion Model proposed in this thesis. We begin by introducing Reservoir Computing, focusing on the Echo State Network (ESN) and its modern, scalable evolution, the Parallel Echo State Network (ParalESN). We then review the evolution of sequence processing paradigms, highlighting the architectural trade-offs between expressivity and computational efficiency. Finally, we detail the framework of Diffusion Language Models, discussing both continuous and discrete approaches, and formalizing the Soft-Masking mechanism that enhances the denoising process.

2.1 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) constitute a foundational class of neural architectures designed for processing sequential data. Unlike feedforward networks, RNNs maintain a hidden state $\mathbf{h}_t \in \mathbb{R}^{N_h}$ that is updated at each time step t as a function of the current input $\mathbf{x}_t \in \mathbb{R}^{N_{in}}$ and the previous hidden state $\mathbf{h}_{t-1}$. In their simplest form, often referred to as *Elman networks* [6], the state transition is given by:

$$\mathbf{h}_t = \sigma(W_h \mathbf{h}_{t-1} + W_{in} \mathbf{x}_t + \mathbf{b}),$$

where $W_h \in \mathbb{R}^{N_h \times N_h}$ is the recurrent weight matrix, $W_{in} \in \mathbb{R}^{N_h \times N_{in}}$ is the input weight matrix, $\mathbf{b} \in \mathbb{R}^{N_h}$ is the bias vector, and $\sigma(\cdot)$ is a pointwise non-linear activation function, typically tanh or ReLU.

The output at each time step is then computed as:

$$\mathbf{y}_t = W_{\text{out}} \mathbf{h}_t + \mathbf{b}_{\text{out}},$$

where $W_{\text{out}} \in \mathbb{R}^{N_{\text{out}} \times N_h}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^{N_{\text{out}}}$ are the output weight matrix and bias, respectively.

Although RNNs are theoretically capable of modeling arbitrarily long-range dependencies, training them in practice is severely hampered by the vanishing and exploding gradient problem [7]. When gradients are propagated through many time steps via backpropagation through time (BPTT) [9], repeated multiplication by W_h causes the gradient magnitudes to either shrink exponentially toward zero or grow unboundedly.

A second fundamental limitation of RNNs is their inherently sequential computation: since each hidden state $\mathbf{h}_t$ depends on the previous state $\mathbf{h}_{t-1}$, the recurrence cannot be parallelized over the temporal dimension. For a sequence of length T , the recurrence requires $O(T)$ sequential steps, severely limiting throughput on modern parallel hardware such as GPUs and TPUs.

These two limitations — gradient instability and sequential processing — motivated the development of gated recurrent architectures and, subsequently, entirely different paradigms for sequence modeling, as discussed in the following sections.

2.2 Reservoir Computing and ParaleSN

While fully trainable models dominate contemporary machine learning, Reservoir Computing (RC) [10] offers an alternative paradigm that avoids Backpropagation Through Time (BPTT) by fixing the internal recurrent weights and relies only on its intrinsic high-dimensional dynamics to process temporal patterns, requiring optimization only at the linear output layer.

2.2.1 Echo State Networks (ESNs)

Echo State Networks (ESNs) [12] are the most prominent RC models. An ESN consists of a high-dimensional, randomly initialized, and untrained recurrent layer (the reservoir) coupled with a simple, trainable linear readout. The state update is given by:

$$\mathbf{h}_t = (1 - \tau)\mathbf{h}_{t-1} + \tau\sigma(W_h \mathbf{h}_{t-1} + W_{\text{in}} \mathbf{x}_t + \mathbf{b})$$

The final prediction is obtained via the linear readout:

$$\mathbf{y}_t = W_{\text{out}} \mathbf{h}_t + \mathbf{b}_{\text{out}}$$

To ensure stability, ESNs rely on the **Echo State Property (ESP)** [12]. Let $\mathbf{x}(t) \in \mathbb{R}^N$ be the state of the reservoir at time t under an input sequence $\mathbf{u} = (\mathbf{u}(t))_{t \in \mathbb{Z}}$. The network possesses the ESP if, for any pair of initial states $\mathbf{x}_0, \mathbf{y}_0 \in \mathbb{R}^N$, the asymptotic states converge:

$$\lim_{t \rightarrow \infty} \|\mathbf{x}(t; \mathbf{x}_0, \mathbf{u}) - \mathbf{x}(t; \mathbf{y}_0, \mathbf{u})\| = 0$$

where $\mathbf{x}(t; \mathbf{x}_0, \mathbf{u})$ indicates the state at time t starting from $\mathbf{x}_0$. Although extremely efficient to train (as only the readout weights W_{out} are optimized, typically via Ridge regression), ESNs are bound by the same sequential processing limitations as standard RNNs, restricting their scalability to long contexts.

2.2.2 Parallel Echo State Networks (ParalESN)

To overcome the sequential constraints and memory footprint of traditional high-dimensional reservoirs, the Parallel Echo State Network (ParalESN) [13] leverages the principles of linear recurrence and complex state spaces inspired by SSMs.

ParalESN replaces the dense, non-linear reservoir with a diagonal linear recurrence in the complex domain. For a given layer l , the state update at time t is expressed as:

$$\mathbf{h}_t^l = (1 - \tau^l) \mathbf{h}_{t-1}^l + \tau^l (\Lambda_h^l \mathbf{h}_{t-1}^l + W_{\text{in}}^l \mathbf{z}_t^{l-1} + \mathbf{b}^l)$$

where:

- $\mathbf{h}_t^l \in \mathbb{C}^{N_h}$ is the complex reservoir state;
- $\mathbf{z}_t^{l-1} \in \mathbb{R}^{N_h}$ is the input from the previous layer's mixing step (or the external input $\mathbb{R}^{N_{\text{in}}}$ for $l = 1$);
- $\Lambda_h^l \in \mathbb{C}^{N_h \times N_h}$ is a diagonal complex transition matrix;
- W_{in}^l is the input weight matrix (dense for the first layer, with a ring topology for subsequent layers to minimize memory overhead);
- $\mathbf{b}^l \in \mathbb{C}^{N_h}$ is the bias vector;
- $\tau^l \in (0, 1]$ is the leaky integration rate.

The effective transition matrix incorporating the leaky rate is

$$\bar{\Lambda}_h^l = (1 - \tau^l)I + \tau^l \Lambda_h^l$$

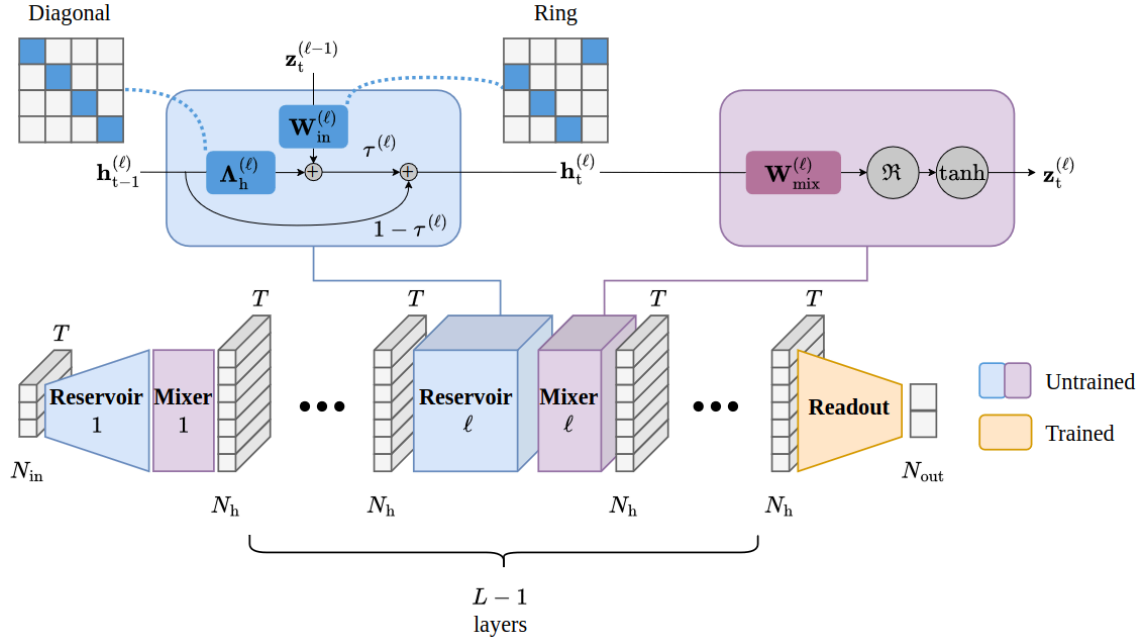


Figure 2.1: ParaleSN consists of multiple blocks, each combining an untrained component (reservoir + non-linear mixing layer) followed by a single trainable readout. The reservoir uses a complex-valued diagonal matrix to process the previous state, allowing the recurrent operation to be parallelized via associative scan. The mixing layer then extracts the real part and passes it through a non-linear activation function to produce an output z . Image from [13]

The diagonal elements $\lambda_i = \rho_i e^{i\theta_i}$ are initialized based on two critical parameters:

- **Spectral Radius (ρ):** Sampled uniformly from $[\rho_{\min}^l, \rho_{\max}^l]$. To satisfy the Echo State Property and guarantee stability, the condition $|\lambda_i| < 1$ is strictly enforced.
- **Phase (θ):** Sampled from $[\theta_{\min}^l, \theta_{\max}^l]$, controlling the frequency of oscillation and enabling the reservoir to capture periodic and complex temporal dynamics.

Because the transition matrix Λ_h^l is diagonal and the recurrence is linear, ParaleSN completely decouples the hidden-state dimensions into parallelizable element-wise operations. This allows the sequence to be processed “unrolled” using an associative scan algorithm, achieving logarithmic time complexity $O(\log L)$ rather than linear $O(L)$.

Input Weight Matrices

For the first layer ($\ell = 1$), the input weight matrix $W_{\text{in}}^{(1)} \in \mathbb{C}^{N_h \times N_{\text{in}}}$ is a dense matrix that maps the external input to the hidden dimension. For deeper layers ($\ell > 1$), a *ring topology* is employed to reduce the memory footprint:

$$W_{\text{in}}^{(\ell>1)} = \begin{pmatrix} 0 & 0 & \cdots & 0 & w_1 \\ w_2 & 0 & \cdots & 0 & 0 \\ 0 & w_3 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & w_{N_h} & 0 \end{pmatrix},$$

which shifts the input vector and applies element-wise scaling, requiring only N_h parameters per layer rather than N_h^2 .

The entries of the input weight matrices are initialized by sampling the real and imaginary parts independently from a uniform distribution over $[-1, 1]$ and then scaling each row i by $\sqrt{1 - |\bar{\lambda}_i|^2}$, where $\bar{\lambda}_i$ is the corresponding effective eigenvalue. The bias vectors are sampled from a uniform distribution and scaled by a hyperparameter $\omega_b^{(\ell)}$.

Parallel Computation via Associative Scan

A key advantage of ParalESN's linear diagonal recurrence is that it can be parallelized over the temporal dimension using the **parallel associative scan** algorithm [14]. Consider the single-layer recurrence in its compact form (dropping layer indices for clarity):

$$\mathbf{h}_t = \bar{\Lambda}_h \mathbf{h}_{t-1} + \mathbf{c}_t,$$

where $\mathbf{c}_t = \tau(W_{\text{in}} \mathbf{z}_t + \mathbf{b})$ collects the input-dependent terms. Since $\bar{\Lambda}_h$ is diagonal, this reduces to N_h independent scalar recurrences $h_{t,i} = \bar{\lambda}_i h_{t-1,i} + c_{t,i}$ for each hidden dimension i . Unrolling the recurrence yields:

$$h_{t,i} = \bar{\lambda}_i^t h_{0,i} + \sum_{k=0}^{t-1} \bar{\lambda}_i^{t-1-k} c_{k+1,i}.$$

This unrolled form reveals that computing all hidden states $\mathbf{h}_1, \dots, \mathbf{h}_T$ corresponds to computing *prefix sums* over the sequence of pairs $(\bar{\lambda}_i, c_{t,i})$ under the binary associative operator:

$$(a_2, b_2) \diamond (a_1, b_1) = (a_2 \cdot a_1, a_2 \cdot b_1 + b_2).$$

This operator is associative (but not commutative), which means the computation can be organized into a balanced binary tree. The parallel associative scan algorithm computes all T prefix results in $O(\log T)$ parallel steps with $O(T)$ total work, reducing the time complexity from the $O(T)$ sequential steps required by the standard recurrence. With $O\left(\frac{T}{\log T}\right)$ parallel processors, the recurrence can be computed in $O(T \log T)$ total work and $O(\log T)$ depth.

Of course, this parallelization relies on $\bar{\Lambda}_h$ being diagonal. If $\bar{\Lambda}_h$ is a dense matrix, the previously described unrolling scheme no longer holds: while diagonal matrices can be represented as vectors to enable parallel execution via prefix-sums, dense matrices require sequential matrix multiplications at each step, breaking the associativity needed for fast unrolling.

Mixing Layer

To introduce necessary non-linearity and combine the independent dimensions of the diagonal reservoir, ParalESN employs a subsequent mixing layer (f_{mix}^l):

$$\mathbf{z}_t^l = f_{\text{mix}}^l(\mathbf{h}_t^l) = \tanh(\Re(W_{\text{mix}}^l * \mathbf{h}_t^l + \mathbf{b}_{\text{mix}}^l))$$

where $W_{\text{mix}}^l \in \mathbb{C}^k$ is a 1D complex convolutional kernel of size k^l sliding along the hidden dimension (with same-padding), $*$ is the 1D convolution operator, and $\Re(\cdot)$ extracts the real part before applying the $\tanh$ activation. The final output is aggregated by the readout layer across all L layers: $\mathbf{y}_t = f_{\text{readout}}(\mathbf{z}_t^1, \dots, \mathbf{z}_t^L)$.

Readout

The readout layer aggregates the mixed states from all L layers to produce the final output:

$$\mathbf{y}_t = f_{\text{readout}}\left(\mathbf{z}_t^{(1)}, \dots, \mathbf{z}_t^{(L)}\right).$$

In the deep configuration, the readout receives the concatenation of mixed states across all layers, providing it with representations at multiple temporal scales. The readout is the *only trainable component* in the ParalESN architecture, consistent with the RC paradigm.

2.3 Gated Recurrent Unit (GRU)

The Gated Recurrent Unit (GRU), introduced by Cho et al. (2014) [16], is a gated variant of the standard RNN designed to alleviate the vanishing gradient problem while maintaining a simpler architecture than the Long Short-Term Memory (LSTM). The GRU employs two gating mechanisms — a reset gate and an update gate — that control the flow of information through the hidden state.

Given an input $\mathbf{x}_t \in \mathbb{R}^{N_{\text{in}}}$ and the previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{N_h}$, the GRU dynamics are defined as:

$$\begin{aligned}\mathbf{r}_t &= \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r), \\ \mathbf{z}_t &= \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z), \\ \tilde{\mathbf{h}}_t &= \tanh(W_h \mathbf{x}_t + U_h (\mathbf{r}_t \circ \mathbf{h}_{t-1}) + \mathbf{b}_h), \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \circ \mathbf{h}_{t-1} + \mathbf{z}_t \circ \tilde{\mathbf{h}}_t,\end{aligned}$$

where $\sigma(\cdot)$ denotes the element-wise sigmoid function, $\circ$ is the Hadamard (element-wise) product, and the weight matrices $W_r, W_z, W_h \in \mathbb{R}^{N_h \times N_{\text{in}}}$ and $U_r, U_z, U_h \in \mathbb{R}^{N_h \times N_h}$ are learnable parameters.

The role of each component is as follows:

- **Reset gate ($\mathbf{r}_t$):** determines how much of the previous hidden state is forgotten when computing the candidate state $\tilde{\mathbf{h}}_t$. When $\mathbf{r}_t \approx 0$, the candidate state is computed almost independently of the previous state, effectively allowing the model to “reset” its memory.
- **Update gate ($\mathbf{z}_t$):** controls the interpolation between the previous hidden state and the candidate state. Values of $\mathbf{z}_t$ close to 1 cause the network to adopt the new candidate state, while values close to 0 preserve the previous state, enabling the model to carry information over long time spans without modification.
- **Candidate state ($\tilde{\mathbf{h}}_t$):** a proposed update to the hidden state, computed from the current input and a reset-modulated version of the previous state.
- **Hidden state ($\mathbf{h}_t$):** the final state obtained as a convex combination of $\mathbf{h}_{t-1}$ and $\tilde{\mathbf{h}}_t$.

The gating mechanism provides the GRU with the ability to learn when to update and when to preserve information, thereby mitigating the vanishing

gradient problem. However, the GRU inherits the sequential computation constraint of standard RNNs: each time step depends on the previous one, resulting in $O(T)$ sequential operations for a sequence of length T . This limitation motivated the search for architectures that can process entire sequences in parallel, as discussed in the following section.

2.4 The Transformer Architecture

The Transformer architecture, introduced by [1], represents a paradigm shift in sequence modeling by entirely dispensing with recurrence. Instead of processing sequences step by step, the Transformer relies on a *self-attention* mechanism that allows each position in the sequence to attend to all other positions simultaneously, enabling full parallelization over the sequence length.

The core operation of the Transformer is **Scaled Dot-Product Attention**. Given a sequence of input representations, three linear projections produce the *query*, *key*, and *value* matrices $Q, K, V \in \mathbb{R}^{L \times d_k}$, where L is the sequence length and d_k is the dimensionality of each head. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where the softmax function is applied row-wise to the matrix $Q \frac{K^\top}{\sqrt{d_k}} \in \mathbb{R}^{L \times L}$. The scaling factor $\sqrt{d_k}$ prevents the dot products from growing excessively large in magnitude, which would push the softmax into regions of extremely small gradients.

In practice, Transformers employ **Multi-Head Attention** (MHA), which runs H attention heads in parallel, each with independent projections $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_k}$, and concatenates the results:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W^O,$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ and $W^O \in \mathbb{R}^{Hd_k \times d_{\text{model}}}$ is the output projection matrix.

A Transformer layer consists of a multi-head self-attention sublayer followed by a position-wise feedforward network (FFN), both wrapped with residual connections and layer normalization:

$$\begin{aligned} \mathbf{h}' &= \mathbf{h} + \text{MHA}(\text{LayerNorm}(\mathbf{h})), \\ \mathbf{h}'' &= \mathbf{h}' + \text{FFN}(\text{LayerNorm}(\mathbf{h}')). \end{aligned}$$

Since self-attention is permutation-equivariant (i.e., it does not inherently encode positional information), Transformers require explicit **positional encodings** to inject information about the order of tokens in the sequence.

The key advantage of the Transformer is its ability to connect any two positions in a single step, unlike RNNs which process sequentially. However, this comes at a quadratic computational cost of $O(L^2)$: as the sequence length L grows, the required computation and memory increase dramatically. To overcome this limitation, variants like linear attention were developed to reduce complexity—though often at the cost of a slight loss in accuracy—driving research toward new architectures like State Space Models.

Flash Linear Attention

Standard Softmax Attention exhibits a quadratic computational complexity $O(N^2)$ with respect to sequence length, creating a severe bottleneck in memory and speed when processing long contexts. **Flash Linear Attention** [17], [18] addresses this issue by completely removing the softmax operator, enabling a recurrent or chunkwise formulation that substantially accelerates computation and improves GPU hardware utilization. Consequently, the computational and memory cost scales linearly $O(N)$ with sequence length. However, this gain in efficiency comes with a trade-off in expressive power and precision: removing the softmax limits the model’s capacity to perform precise token retrieval. In particular, empirical and theoretical findings—such as those on associative recall tasks [19]—demonstrate that for linear attention to match the associative memory capabilities of standard softmax attention, its hidden state dimension must scale proportionally with the sequence length.

2.5 State-Space Models (SSM)

State-Space Models (SSMs) [20] emerged as a promising alternative to overcome the limitations of both RNNs and Transformers. Grounded in continuous dynamical systems, SSMs map a 1-D input sequence $x(t)$ to an output $y(t)$ via a latent state $\mathbf{h}(t)$:

$$\begin{aligned}\frac{d\mathbf{h}(t)}{dt} &= \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t) \\ y(t) &= \mathbf{C}\mathbf{h}(t) + Dx(t)\end{aligned}$$

where $\mathbf{A} \in \mathbb{R}^{N \times N}$ is the state transition matrix governing the dynamics of the latent state, $\mathbf{B} \in \mathbb{R}^{N \times 1}$ is the input projection vector, $\mathbf{C} \in \mathbb{R}^{1 \times N}$ is the output projection vector, and $D \in \mathbb{R}$ is a direct feedthrough scalar (often omitted or set to zero). Here, N denotes the state dimension, which determines the memory capacity of the system.

2.5.1 Discretization

To apply continuous-time SSMs to discrete sequences $x_0, x_1, \dots, x_{T-1}$, the continuous dynamics must be *discretized* with a step size $\Delta > 0$. A common approach is the **Zero-Order Hold (ZOH)** discretization, which assumes the input is constant over each interval $[t, t + \Delta)$. This yields the discrete-time recurrence:

$$\begin{aligned}\mathbf{h}_t &= \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t, \\ y_t &= \mathbf{C}\mathbf{h}_t + Dx_t,\end{aligned}$$

where the discretized matrices are defined as:

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}), \quad \bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I})\Delta\mathbf{B}.$$

A crucial insight is that this discrete-time recurrence can be equivalently expressed as a **global convolution**, since unrolling the recurrence yields:

$$y_t = \mathbf{C}\bar{\mathbf{A}}^t\bar{\mathbf{B}}x_0 + \mathbf{C}\bar{\mathbf{A}}^{t-1}\bar{\mathbf{B}}x_1 + \dots + \mathbf{C}\bar{\mathbf{B}}x_t + Dx_t = \sum_{k=0}^t \bar{\mathbf{K}}_k x_{t-k},$$

where $\overline{K}_k = \mathbf{C}\overline{\mathbf{A}}^k\overline{\mathbf{B}}$ is the SSM convolution kernel. This convolutional view enables efficient parallel computation during training.

2.5.2 S5

A critical component of early successful SSMs is the HiPPO (High-Order Polynomial Projection Operators) initialization, introduced in [21], [22], which initializes the matrix A to memorize the history of a sequence optimally. For instance, the HiPPO-LegS matrix is defined as:

$$A_{nk} = \begin{cases} -\sqrt{(2n+1)(2k+1)} & \text{if } n > k \\ -(n+1) & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}$$

The S5 architecture [23] simplifies standard SSMs by diagonalizing the state matrix and applying the S4D approach [24], which extracts the complex eigenvalues from the HiPPO matrix. Because the recurrence in these model is entirely linear, the sequential computation can be parallelized using the associative scan (prefix-sum) algorithm, reducing the temporal complexity from $O(L)$ to $O(\log L)$ and breaking the sequential bottleneck of traditional RNNs.

2.5.3 Linear Recurrent Units (LRU)

The LRU (Linear Recurrent Unit) model [25] redefines deep sequence modeling by casting traditional recurrent layers into purely linear, diagonal state-space dynamics. Unlike earlier State Space Models (SSMs), such as S5 which strictly relied on HiPPO initialization, LRU abandons HiPPO entirely in favor of an initialization scheme in the discrete complex plane.

Specifically, the state transition matrix $A \in \mathbb{C}^{N \times N}$ is constrained to be diagonal, parameterized via polar coordinates $A = \text{diag}(\nu \circ e^{i\theta})$ to explicitly control state stability through magnitude bounds $\nu \in (0, 1)$.

By decoupling sequence length from recurrent execution, also LRU models, as for S5, leverage *parallel scan* algorithms to compute all hidden states in $O(N \log L)$ parallel time during training, combining the fast parallelization of Transformers with the $O(1)$ constant-memory inference of classical RNNs.

2.5.4 MAMBA

While linear, time-invariant SSMs scale efficiently, they struggle with tasks requiring dynamic, input-dependent context selection. MAMBA [26] addresses this by introducing a selective state-space mechanism where the parameters $\mathbf{B}$, $\mathbf{C}$, and Δ are functions of the input x_t :

$$\mathbf{B}_t = \text{Linear}_N(x_t)$$

$$\mathbf{C}_t = \text{Linear}_N(x_t)$$

$$\Delta_t = \text{softplus}(\text{Parameter} + \text{Linear}_1(x_t))$$

This input-dependent selection allows MAMBA to filter irrelevant information and remember crucial context, achieving state-of-the-art performance. However, this selectivity sacrifices time-invariance, meaning the recurrence can no longer be computed using a standard associative scan, requiring custom hardware-aware prefix-sum implementations to maintain efficiency. This computational bottleneck is addressed by MAMBA-2[27], which introduces the State Space Duality (SSD) framework, bridging structured state-space models with attention mechanism dynamics and enabling the use of high-performance matrix multiplication algorithms on modern GPUs.

2.6 Diffusion Models

Diffusion models are the generative framework on which the model proposed in this thesis is built. For this reason, this section introduces them step

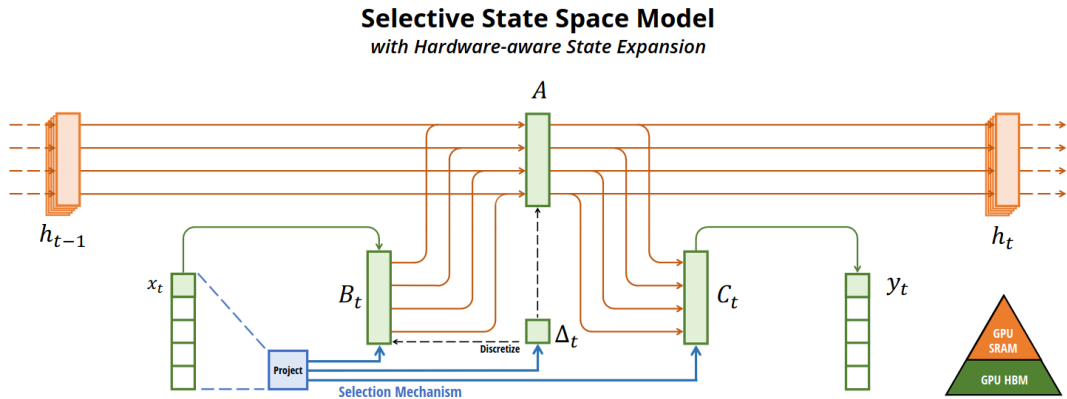


Figure 2.2: The selective architecture of MAMBA. The blue rectangle describes the Selection Mechanism, which takes the input x_t and projects it to compute $\mathbf{B}_t$, $\mathbf{C}_t$, and Δ_t . Image adapted from [26].

by step, moving from the general idea to the specific case of text. We first describe how diffusion models work, both intuitively and formally, independently of the type of data. We then discuss why diffusion is an interesting alternative to autoregressive models for text generation, and why applying it to language is not straightforward: unlike images, text is made of discrete symbols. This leads to two families of approaches, *continuous* diffusion models (Section 2.6.1) and *discrete* diffusion models (Section 2.6.2), whose main differences are summarized at the end of this introduction. Masked Diffusion Language Models, in particular, are the direct foundation of the Soft-Masking mechanism (Section 2.6.3) and of the DiESN model presented in Chapter 3.

The Intuition: Generation as Iterative Denoising

The key observation behind diffusion models [28], [29] is that destroying the structure of data is easy, while creating it is hard. If we take an image and repeatedly add a small amount of random noise to it, after enough steps the result is indistinguishable from pure noise, and no knowledge about images is needed to do this. Generating a realistic image from scratch, on the other hand, is very difficult. Diffusion models connect these two directions: they learn to *undo* the corruption, one small step at a time. If a neural network knows how to turn a slightly noisy sample into a slightly cleaner one, then it can generate new data by starting from pure noise and applying this denoising step many times.

A diffusion model is therefore made of two processes, illustrated in Figure 2.3:

- The **forward pass (corruption)** gradually transforms a clean data sample $\mathbf{x}_0$ into a sequence of increasingly noisy versions $\mathbf{x}_1, \dots, \mathbf{x}_T$. After T steps, $\mathbf{x}_T$ contains almost no information about $\mathbf{x}_0$ and follows a simple, known distribution, such as a standard Gaussian.
- The **reverse pass (denoising)** goes in the opposite direction, from $\mathbf{x}_T$ back to $\mathbf{x}_0$. Each reverse step is performed by a neural network trained to remove a small amount of noise. To generate new data, we sample $\mathbf{x}_T$ from the simple distribution and apply the learned reverse steps until we obtain a clean sample.

The network never has to produce a complete sample in a single shot: it only has to make its input slightly cleaner. The difficult task of generation is thus split into many simple denoising tasks, all solved by the same network,

which receives the step t (or, equivalently, the noise level) as an additional input, so that it knows how much noise is present in its input.

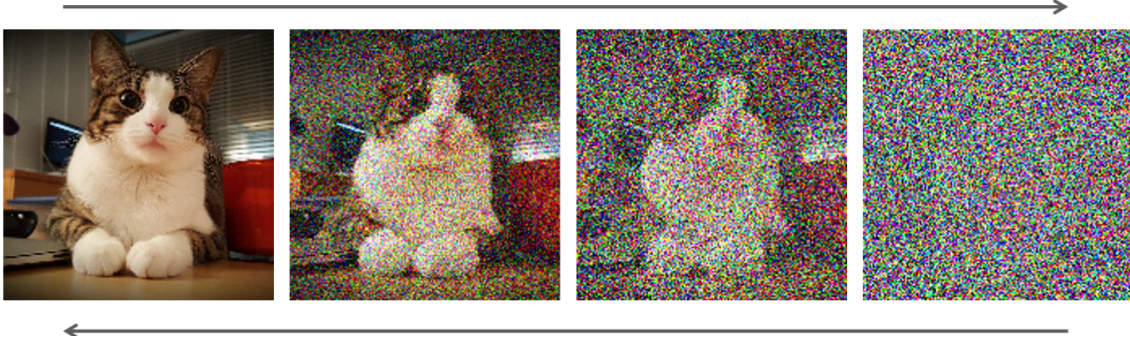


Figure 2.3: **The diffusion process:** Starting from a clean image, random noise is progressively added until the original structure is completely obscured (forward pass). During generation, the process is reversed: the model gradually removes the noise, step by step, to reconstruct the image (backward pass).

Formal Definition

Formally, the forward process is a Markov chain that progressively transforms data into noise:

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1}),$$

where $\mathbf{x}_0 \sim q_{\text{data}}(\mathbf{x}_0)$ is drawn from the data distribution and $q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$ defines the transition probability at step t . The Markov property means that each $\mathbf{x}_t$ depends only on the previous step $\mathbf{x}_{t-1}$. The transitions are defined by a *noise schedule*, which decides how much noise is added at each step, and they are chosen so that two properties hold. First, the marginal distribution $q(\mathbf{x}_t \mid \mathbf{x}_0)$ is available in closed form, so that a noisy version $\mathbf{x}_t$ can be sampled directly from $\mathbf{x}_0$ at any step, without simulating the whole chain. Second, at the last step the data is completely destroyed, so that $q(\mathbf{x}_T \mid \mathbf{x}_0) \approx p(\mathbf{x}_T)$, where $p(\mathbf{x}_T)$ is a fixed *prior* distribution that does not depend on $\mathbf{x}_0$.

The reverse process aims to reconstruct the original data using a parameterized model:

$$p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t).$$

It starts from the prior $p(\mathbf{x}_T)$ and applies the learned transitions $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$, each implemented by a neural network with parameters θ . Ideally, each $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ should match the true reverse transition $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$. Unfortunately, this distribution is intractable, because computing it would require knowledge of the entire data distribution. However, it becomes tractable if we also condition on the clean sample $\mathbf{x}_0$. Using Bayes' rule and the Markov property, we obtain the *posterior*:

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1}) q(\mathbf{x}_{t-1} | \mathbf{x}_0)}{q(\mathbf{x}_t | \mathbf{x}_0)},$$

where all three terms on the right-hand side are known. This posterior describes what a correct denoising step looks like when the clean data is known, and it is used as the target during training.

Training objective. Ideally, the model would be trained by maximizing the likelihood $p_\theta(\mathbf{x}_0) = \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T}$ of the training data, but this integral over all possible noisy trajectories is intractable. As in variational inference, we instead minimize an upper bound on the negative log-likelihood, known as the Negative Evidence Lower Bound (NELBO):

$$-\log p_\theta(\mathbf{x}_0) \leq \mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right] = \mathcal{L}_{\text{NELBO}}.$$

By exploiting the Markov structure of both processes, this bound can be rewritten as a sum of simple terms [28], [29]:

$$\mathcal{L}_{\text{NELBO}} = \mathbb{E}_q \left[\mathcal{L}_T + \sum_{t=2}^T \mathcal{L}_{t-1} + \mathcal{L}_0 \right],$$

with

$$\begin{aligned} \mathcal{L}_T &= D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T)), \\ \mathcal{L}_{t-1} &= D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)), \\ \mathcal{L}_0 &= -\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1), \end{aligned}$$

where D_{KL} denotes the Kullback-Leibler divergence. Each term has a clear meaning:

- $\mathcal{L}_T$ (*prior matching*) measures how close the end of the forward process is to the prior. It has no trainable parameters and is close to zero by construction.

- $\mathcal{L}_{t-1}$ (*denoising matching*) compares, at every step, the learned reverse transition with the tractable posterior. Minimizing it teaches the network what $\mathbf{x}_{t-1}$ should look like, given $\mathbf{x}_t$.
- $\mathcal{L}_0$ (*reconstruction*) measures how well the last reverse step recovers the clean data.

Parameterization of the denoiser. Instead of asking the network to output $\mathbf{x}_{t-1}$ directly, it is usually more convenient to let it predict the clean data $\mathbf{x}_0$ from the noisy input, and then to plug this prediction into the posterior:

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0 = f_\theta(\mathbf{x}_t, t)),$$

where $f_\theta(\mathbf{x}_t, t)$ is the neural network, called the *denoiser*, which takes the noisy sample and the current step and returns an estimate of $\mathbf{x}_0$. With this choice, the network always solves the same kind of problem, namely “guess the clean data from its corrupted version”, while the step t tells it how difficult the problem is. This parameterization is used by both continuous and discrete diffusion models, and it is exactly the one adopted by MDLMs (Section 2.6.2).

Sampling. Once the denoiser is trained, new samples are generated by running the reverse chain:

1. sample $\mathbf{x}_T \sim p(\mathbf{x}_T)$;
2. for $t = T, T-1, \dots, 1$: compute the prediction $\hat{\mathbf{x}}_0 = f_\theta(\mathbf{x}_t, t)$ and sample $\mathbf{x}_{t-1} \sim q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \hat{\mathbf{x}}_0)$;
3. return $\mathbf{x}_0$.

The number of steps T controls a trade-off between quality and speed: more steps make each denoising task easier, but require more evaluations of the network.

Continuous time. Many recent models, including MDLM, describe the process in continuous time, with $t \in [0, 1]$: $t = 0$ corresponds to clean data and $t = 1$ to pure noise. In this view, the T steps of the chain are a discretization of the interval $[0, 1]$, and a single reverse step moves from a time t to an earlier time $s = t - 1/T$. As $T \rightarrow \infty$, the sum in the NELBO becomes an integral over t . The ideas presented above remain unchanged; therefore, in the following we write $\mathbf{x}_{t-1}$ (discrete steps) or $\mathbf{x}_s$ with $s < t$ (continuous time), depending on the context.

From Images to Text

Diffusion models first became successful in continuous domains such as images and audio. Their application to text is motivated by the limitations of the autoregressive (AR) paradigm that dominates language modeling. Let a sentence be a sequence of L tokens $\mathbf{x} = (\mathbf{x}^1, \dots, \mathbf{x}^L)$, where each token belongs to a finite vocabulary $\mathcal{V}$. An AR model factorizes the probability of the sequence with the chain rule:

$$p_{\theta}(\mathbf{x}) = \prod_{l=1}^L p_{\theta}(\mathbf{x}^l \mid \mathbf{x}^1, \dots, \mathbf{x}^{l-1}),$$

and generates text one token at a time, from left to right, as shown in Figure 2.4 (a). This approach is very effective, but it has some structural drawbacks:

- generating L tokens requires L sequential evaluations of the network, which cannot be parallelized;
- each token is chosen by looking only at the tokens on its left and, once generated, it is never revised, so early mistakes propagate to the rest of the sequence;
- the fixed left-to-right order makes tasks such as filling in missing parts of a text, or controlling global properties of the output, less natural.

In the context of language modeling, diffusion-based approaches offer a compelling alternative to this paradigm. As shown in Figure 2.4 (b), a diffusion language model starts from a fully corrupted sequence of length L and refines all positions in parallel over a number of denoising steps. At each step, the denoiser looks at the whole sequence, using the context on both sides of each token, and it can update many tokens at once. Since the number of steps can be smaller than the sequence length, generation may require fewer network evaluations than AR decoding, and the number of steps gives a direct way to trade quality for speed. An important consequence for this thesis is that the denoiser is not causal: like a BERT encoder [30], it must read the whole sequence in both directions. This is the reason why the DiESN model presented in Chapter 3 relies on a bidirectional recurrent backbone.

The main difficulty is that text is discrete. Each token is a symbol of $\mathcal{V}$, which we represent as a one-hot vector $\mathbf{x}^l \in \{0, 1\}^{|\mathcal{V}|}$. The diffusion framework was originally designed for continuous data, where corruption consists of adding small Gaussian perturbations. For text, this idea does

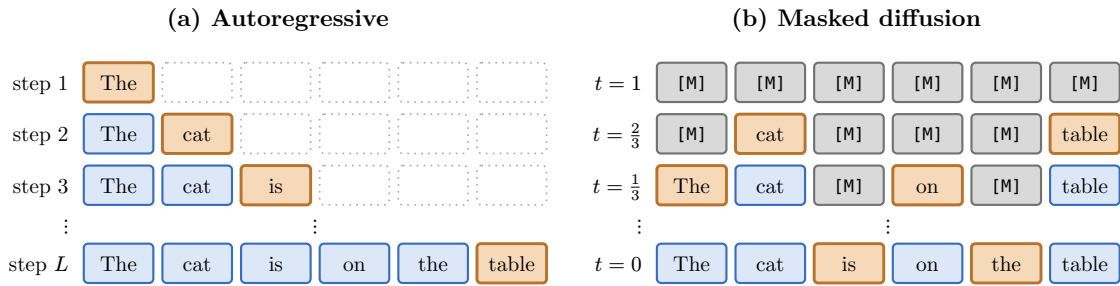


Figure 2.4: Autoregressive generation versus diffusion-based generation of the same sentence. (a) An AR model generates one token per step, from left to right, and never revises a token once it has been produced. (b) A masked diffusion model starts from a sequence of [MASK] tokens and, at each denoising step, reveals several tokens in parallel, using the context on both sides. Highlighted tokens are the ones produced at the current step.

not apply directly: adding “a little noise” to the word *cat* does not produce another word, and there is no natural notion of a token that is “slightly noisy”. Two main strategies have been proposed to solve this problem, and they define the two families of diffusion language models.

Continuous versus Discrete Diffusion

For language, the key distinction lies in how the corruption is performed (Figure 2.5):

- **Continuous diffusion models** operate in the embedding space. Each token is first mapped to a continuous vector (its embedding), and standard Gaussian noise is added to these vectors. At the end of the reverse process, the denoised vectors must be mapped back to tokens, a step called *rounding*.
- **Discrete diffusion models**, by contrast, operate directly on the discrete token space. They corrupt tokens through stochastic transitions (e.g., replacing tokens with random tokens or with a special [MASK] token), described by categorical distributions, and reverse this corruption through categorical predictions over the vocabulary. As a result, every intermediate state is a valid sequence of tokens.

In both cases the general framework is the same: a fixed forward process, a learned reverse process, and a training objective derived from the NELBO. What changes is the nature of the noise. In the continuous case, noise is a small and smooth perturbation of a vector; in the discrete case, it is a sudden

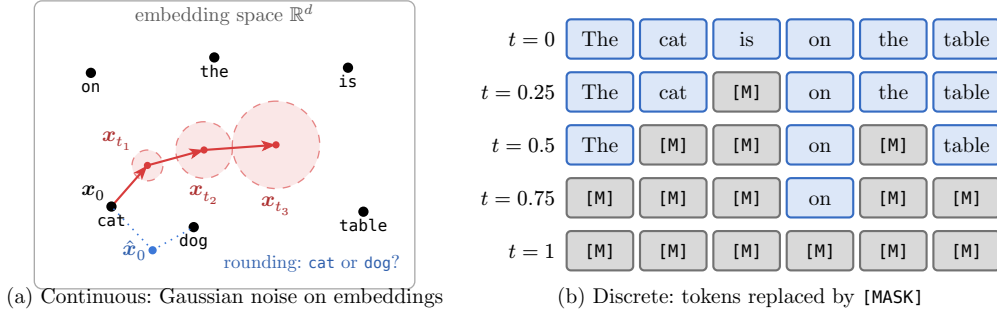


Figure 2.5: Continuous and discrete corruption of text. (a) In continuous diffusion, the embedding $\mathbf{x}_0$ of the token *cat* is progressively perturbed with Gaussian noise of increasing variance (dashed circles) and drifts away from the embeddings of real words. At the end of the reverse process, the predicted vector $\hat{\mathbf{x}}_0$ must be rounded to the closest word embedding, which can be ambiguous. (b) In discrete (masked) diffusion, each token is kept or replaced by [MASK]; the fraction of masked tokens grows with t , and every intermediate state is still a sequence of tokens.

jump from one token to another. The main differences are summarized in Table 2.1.

	Continuous diffusion	Discrete diffusion
Token representation	embedding vector in $\mathbb{R}^d$	one-hot vector in $\{0, 1\}^{ \mathcal{V} }$
Forward corruption	Gaussian noise	replace the token (random token or [MASK])
Prior $p(\mathbf{x}_T)$	$\mathcal{N}(\mathbf{0}, I)$	uniform over $\mathcal{V}$, or all [MASK]
Denoiser output	noise $\boldsymbol{\varepsilon}$ or clean embedding	probability distribution over $\mathcal{V}$
Training loss	mean squared error	(weighted) cross-entropy
Back to tokens	rounding step required	not required
Examples	Diffusion-LM [31]	D3PM [32], MDLM [33]

Table 2.1: Main differences between continuous and discrete diffusion models for text.

Neither approach is perfect. Continuous models can reuse the well-developed tools of Gaussian diffusion, and their continuous state can carry fine-grained information, such as uncertainty between similar words; however, they must bridge the gap between vectors and tokens. Discrete models match the nature of text, but their corruption and denoising steps are hard decisions

on individual tokens. As we will see, Soft-Masking (Section 2.6.3) can be interpreted as a way to bring part of this continuous information back into a discrete masked diffusion model. The next two subsections describe the two families in more detail.

2.6.1 Continuous Diffusion Modeling

The most widely used continuous formulation is the Denoising Diffusion Probabilistic Model (DDPM) [29]. We first present it in its general form, and then describe how it has been adapted to text.

Gaussian Diffusion

Standard diffusion models learn to reverse a Markovian forward process that gradually corrupts data with Gaussian noise. Each forward step slightly shrinks the current sample and adds a small amount of noise:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t I),$$

where $\beta_t \in (0, 1)$ is the variance of the noise added at step t , usually small and increasing with t . Defining $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$, the Gaussian transitions can be composed in closed form:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t)I).$$

In practice, this means that the noisy sample at any step can be obtained in a single operation:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon},$$

where $\mathbf{x}_0$ is the clean data, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, I)$ is the added noise, and $\bar{\alpha}_t$ is the parameter of the noise schedule—strictly decreasing from 1 to 0 over time—that defines how much residual signal remains at time t . Since $\bar{\alpha}_T \approx 0$, the last sample is essentially pure noise, and the prior is $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, I)$.

The posterior of the forward process is also Gaussian:

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t I),$$

with

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t.$$

In other words, the mean of a correct denoising step is a weighted average of the clean sample $\mathbf{x}_0$ and the current noisy sample $\mathbf{x}_t$. Accordingly, the reverse transitions are also modeled as Gaussians with a learned mean, $p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 I)$, where the variance σ_t^2 is fixed (e.g., $\sigma_t^2 = \tilde{\beta}_t$). Since both distributions are Gaussian, each denoising matching term of the NELBO reduces to a squared distance between their means:

$$\mathcal{L}_{t-1} = \frac{1}{2\sigma_t^2} \|\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_\theta(\mathbf{x}_t, t)\|^2 + C,$$

where C does not depend on θ . Ho et al. [29] observed that, by substituting $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\varepsilon})/\sqrt{\bar{\alpha}_t}$, the posterior mean can be written in terms of the noise, $\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\boldsymbol{\varepsilon})$. It is then natural to use a neural network $\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)$ that predicts the noise contained in $\mathbf{x}_t$, and to set:

$$\boldsymbol{\mu}_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) \right).$$

Predicting $\boldsymbol{\varepsilon}$ is equivalent to predicting $\mathbf{x}_0$, since, given $\mathbf{x}_t$, one can be computed from the other. With this choice, $\mathcal{L}_{t-1}$ becomes a weighted squared error between the true and the predicted noise. Dropping the weights leads to the simplified training objective:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\varepsilon}} [\|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t)\|^2].$$

In words, the network is shown a clean sample corrupted with a known amount of noise, and it learns to recognize which noise was added. Once trained, the backward process utilizes $\boldsymbol{\varepsilon}_\theta$ to iteratively denoise $\mathbf{x}_t$ step-by-step back to $\mathbf{x}_0$: starting from $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, I)$, it repeatedly computes

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, I),$$

for $t = T, \dots, 1$ (with $\mathbf{z} = \mathbf{0}$ at the last step). Interestingly, the predicted noise is also related to the gradient of the log-density of the noisy data,

$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t) \approx -\varepsilon_\theta(\mathbf{x}_t, t) / \sqrt{1 - \bar{\alpha}_t}$, which connects DDPM to score-based generative models [34]: each reverse step moves the sample towards regions where the data is more likely.

Continuous Diffusion for Text

Applying continuous diffusion directly to text [31] involves mapping discrete tokens to continuous embeddings. In Diffusion-LM [31], each token w^l of a sentence $\mathbf{w} = (w^1, \dots, w^L)$ is mapped to a learned vector $\text{Emb}(w^l) \in \mathbb{R}^d$, so that the whole sentence becomes a matrix $\text{Emb}(\mathbf{w}) \in \mathbb{R}^{L \times d}$, on which Gaussian diffusion can operate as described above. Two additional steps connect the discrete and the continuous worlds:

- an **embedding step**, added at the beginning of the forward process, $q_\varphi(\mathbf{x}_0 \mid \mathbf{w}) = \mathcal{N}(\mathbf{x}_0; \text{Emb}(\mathbf{w}), \sigma_0^2 I)$, which turns the discrete sentence into a continuous starting point;
- a **rounding step**, added at the end of the reverse process, $p_\theta(\mathbf{w} \mid \mathbf{x}_0) = \prod_{l=1}^L p_\theta(w^l \mid \mathbf{x}_0^l)$, a softmax over the vocabulary that maps each denoised vector back to a token.

The embeddings are learned jointly with the denoiser by minimizing the end-to-end objective:

$$\mathcal{L}_{\text{e2e}}(\mathbf{w}) = \mathbb{E}_{q_\varphi(\mathbf{x}_{0:T} \mid \mathbf{w})} \left[\mathcal{L}_{\text{simple}}(\mathbf{x}_0) + \|\text{Emb}(\mathbf{w}) - \boldsymbol{\mu}_\theta(\mathbf{x}_1, 1)\|^2 - \log p_\theta(\mathbf{w} \mid \mathbf{x}_0) \right],$$

where the last term is a cross-entropy loss that keeps the embeddings of different tokens distinguishable, so that rounding remains possible.

In practice, rounding is the weak point of this approach. At the end of the reverse process, the vectors $\mathbf{x}_0$ do not necessarily lie on a word embedding: they can fall between the embeddings of different tokens, as shown in Figure 2.5 (a), and the rounding step may then choose the wrong word. To reduce this problem, Diffusion-LM trains the network to predict $\mathbf{x}_0$ directly and, at each sampling step, replaces this prediction with the nearest word embedding (the *clamping trick*), forcing the intermediate states to commit to valid tokens. Despite these solutions, reconstructing discrete text from a noisy continuous vector often leads to significant rounding errors and representation mismatch, severely hindering the model’s self-correction capabilities. Moreover, the training loss is a distance in the embedding space, which is only indirectly related to the likelihood of the text, and a large number of denoising steps is typically needed. Later works improved

continuous diffusion for language, for example by training the denoiser with a cross-entropy loss [35] or by carefully designing likelihood-based training recipes [36], but a gap in likelihood with autoregressive models of comparable size remained. These limitations motivated the development of diffusion models that operate directly in the discrete space of tokens.

2.6.2 Masked Diffusion Language Models

To circumvent the limitations of continuous embeddings, discrete diffusion models define the corruption process directly on tokens. We first describe the general framework of discrete diffusion based on transition matrices, introduced by D3PM [32], and then its masked variant, the Masked Diffusion Language Model (MDLM) [33], which is the formulation adopted in this thesis.

Discrete Diffusion with Transition Matrices

Since, in all the models discussed here, the tokens of a sequence are corrupted independently, we can describe the forward process for a single token. Let $\mathbf{x} \in \{0, 1\}^{|\mathcal{V}|}$ be the one-hot (row) vector of a token, and let $\text{Cat}(\mathbf{x}; \mathbf{p})$ denote the categorical distribution over tokens with probability vector $\mathbf{p}$. A forward step is defined by a transition matrix $\mathbf{Q}_t \in [0, 1]^{|\mathcal{V}| \times |\mathcal{V}|}$:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \text{Cat}(\mathbf{x}_t; \mathbf{x}_{t-1} \mathbf{Q}_t),$$

where $[Q_t]_{ij} = q(x_t = j \mid x_{t-1} = i)$ is the probability that token i becomes token j , so that each row of $\mathbf{Q}_t$ sums to one. Composing several steps simply amounts to multiplying the matrices. Defining $\bar{\mathbf{Q}}_t = \mathbf{Q}_1 \mathbf{Q}_2 \cdots \mathbf{Q}_t$, the marginal and the posterior of the forward process are:

$$\begin{aligned} q(\mathbf{x}_t \mid \mathbf{x}_0) &= \text{Cat}(\mathbf{x}_t; \mathbf{x}_0 \bar{\mathbf{Q}}_t), \\ q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) &= \text{Cat}\left(\mathbf{x}_{t-1}; \frac{\mathbf{x}_t \mathbf{Q}_t^\top \circ \mathbf{x}_0 \bar{\mathbf{Q}}_{t-1}}{\mathbf{x}_0 \bar{\mathbf{Q}}_t \mathbf{x}_t^\top}\right), \end{aligned}$$

where $\circ$ is the element-wise product. Discrete diffusion therefore has exactly the same ingredients as Gaussian diffusion: closed-form marginals and a tractable posterior. The NELBO derived above applies unchanged, with KL divergences between categorical distributions instead of Gaussian ones. The denoiser $f_\theta(\mathbf{x}_t, t)$ now outputs, for each position, a probability distribution

over the vocabulary (through a softmax), which replaces $\mathbf{x}_0$ in the posterior to obtain the reverse transitions.

The choice of Q_t determines how tokens are corrupted. The most effective way is by extending the vocabulary to include a special [MASK] token. During the transition process, each token has a probability $1 - \beta_t$ of remaining unchanged and a probability β_t of being replaced by the [MASK] token. In this case $Q_t = (1 - \beta_t)I + \beta_t \mathbf{1m}$.

Austin et al. [32] found that this process works best for text. This corruption closely resembles the masked language modeling task of BERT [30]: the denoiser receives a sentence in which some words are hidden and must guess them from the visible context.

Masked Diffusion Language Models (MDLM)

MDLM [33] formulates this diffusion process in continuous time, $t \in [0, 1]$. Instead of using step-wise matrices, the forward process is described directly by its marginal: tokens are independently absorbed into the [MASK] state according to

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \text{Cat}(\mathbf{x}_t; \alpha_t \mathbf{x}_0 + (1 - \alpha_t) \mathbf{m}),$$

that is, each token is left unchanged with probability α_t and is replaced by [MASK] with probability $1 - \alpha_t$. The function α_t is the noise schedule: it is strictly decreasing from $\alpha_0 \approx 1$ (clean data) to $\alpha_1 \approx 0$ (fully masked sequence), and it represents the probability that a token is still visible at time t . It is often written as $\alpha_t = e^{-\sigma(t)}$, where $\sigma(t)$ is the total noise level at time t ; this is the quantity used to condition the denoiser in Chapter 3. A common choice, adopted by MDLM and also used in this thesis, is the log-linear noise schedule $\sigma(t) = -\log(1 - (1 - \varepsilon)t)$, i.e., $\alpha_t = 1 - (1 - \varepsilon)t$, where ε is a small constant: with this schedule, the expected fraction of masked tokens grows linearly with t .

For $s < t$, the posterior of this process has a very simple form:

$$q(\mathbf{x}_s \mid \mathbf{x}_t, \mathbf{x}_0) = \begin{cases} \text{Cat}(\mathbf{x}_s; \mathbf{x}_t) & \text{if } \mathbf{x}_t \neq \mathbf{m} \\ \text{Cat}\left(\mathbf{x}_s; \frac{(1 - \alpha_s)\mathbf{m} + (\alpha_s - \alpha_t)\mathbf{x}_0}{1 - \alpha_t}\right) & \text{if } \mathbf{x}_t = \mathbf{m}. \end{cases}$$

Intuitively, a token that is visible at time t was also visible at every earlier time, so it stays unchanged. A token that is masked at time t , instead, was either already masked at time s , with probability $(1 - \alpha_s)/(1 - \alpha_t)$,

or still visible and equal to the original token $\mathbf{x}_0$, with probability $(\alpha_s - \alpha_t)/(1 - \alpha_t)$.

Reverse process. During generation $\mathbf{x}_0$ is unknown, so it is replaced by the prediction of the denoiser $f_\theta(\mathbf{x}_t, t)$, which gives a probability distribution over the vocabulary for each position. MDLM adds two simple constraints to this prediction, known as the SUBS parameterization:

- *zero masking probabilities*, meaning that the denoiser never predicts [MASK] as a clean token, since clean data contains no masks;
- *carry-over unmasking*, meaning that a token that is already visible in $\mathbf{x}_t$ is simply copied. The resulting reverse transition for each token is:

$$p_\theta(\mathbf{x}_s | \mathbf{x}_t) = \begin{cases} \text{Cat}(\mathbf{x}_s; \mathbf{x}_t) & \text{if } \mathbf{x}_t \neq \mathbf{m} \\ \text{Cat}\left(\mathbf{x}_s; \frac{\alpha_s - \alpha_t}{1 - \alpha_t} f_\theta(\mathbf{x}_t, t) + \frac{1 - \alpha_s}{1 - \alpha_t} \mathbf{m}\right) & \text{if } \mathbf{x}_t = \mathbf{m} \end{cases}$$

where $\mathbf{m}$ represents the [MASK] token and α_t serves as the noise schedule parameter. Note that the prediction for a given position depends on the whole noisy sequence $\mathbf{x}_t$, and not only on the token at that position: this is how the context is used to recover the missing words.

Training objective. By plugging this reverse process into the NELBO and taking the continuous-time limit ($T \rightarrow \infty$), Sahoo et al. [33] show that the bound reduces to:

$$\mathcal{L}_{\text{NELBO}} = \mathbb{E}_q \int_0^1 \frac{\alpha'_t}{1 - \alpha_t} \sum_{l=1}^L \log \langle f_\theta^l(\mathbf{x}_t, t), \mathbf{x}_0^l \rangle dt,$$

where $\alpha'_t = d\alpha_t/dt < 0$, $f_\theta^l(\mathbf{x}_t, t)$ is the distribution predicted for position l , and $\langle f_\theta^l(\mathbf{x}_t, t), \mathbf{x}_0^l \rangle$ is the probability that it assigns to the correct token. Because of the carry-over constraint, visible tokens contribute $\log 1 = 0$, so only masked positions matter. With the log-linear schedule, $\alpha'_t/(1 - \alpha_t) = -1/t$, and the objective becomes a weighted cross-entropy:

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim U(0,1), \mathbf{x}_0 \sim q_{\text{data}}, \mathbf{x}_t \sim q(\mathbf{x}_t | \mathbf{x}_0)} \left[\frac{1}{t} \sum_{l=1}^L \mathbf{1}_{\mathbf{x}_t^l = \mathbf{m}} (-\log \langle f_\theta^l(\mathbf{x}_t, t), \mathbf{x}_0^l \rangle) \right].$$

This objective has a simple interpretation. At each training step we (1) sample a random time t , (2) mask each token of a training sentence with probability approximately equal to t , (3) ask the denoiser to recover the masked tokens, and (4) compute the cross-entropy loss on the masked positions only. The factor $1/t$ compensates for the small number of masked

tokens at low noise levels and makes the loss a valid upper bound on the negative log-likelihood, so that the perplexity of the model can be estimated and compared with that of AR models. This is the objective used to train DiESN (Section 3.6).

Sampling. Generation runs the reverse process from $t = 1$ to $t = 0$ in T steps. Starting from a sequence made only of [MASK] tokens, each step from time t to time $s = t - 1/T$ works as follows:

1. a single forward pass of the denoiser computes $f_\theta(\mathbf{x}_t, t)$ for all positions in parallel;
2. each masked position is unmasked with probability $(\alpha_s - \alpha_t)/(1 - \alpha_t)$, in which case its token is sampled from the predicted distribution; otherwise it remains masked;
3. visible tokens are copied unchanged.

At the last step, all the remaining masked tokens are revealed. One such step is illustrated in Figure 2.6.

In standard MDLMs, this de-noising therefore involves a **binary decision**: for each masked token, the model either retains the mask or replaces it entirely with a single token sampled from the predicted distribution $f_\theta(\mathbf{x}_t, t)$. Moreover, because of the carry-over constraint, a token that has been unmasked stays fixed until the end of generation. This discrete binary step forces a hard commitment and discards the rich probability distribution computed by the model, preventing partial information or uncertainty from

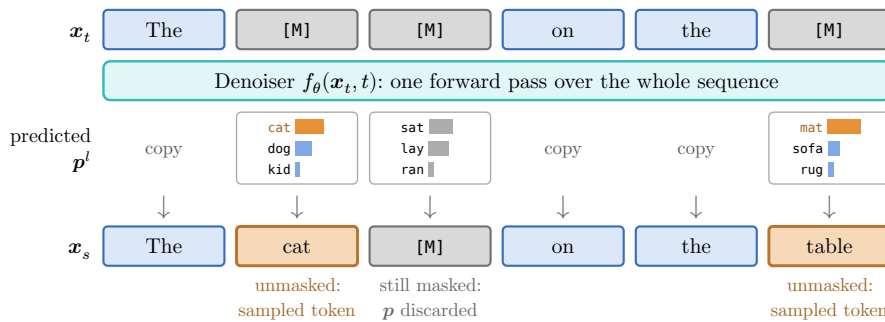


Figure 2.6: One reverse step of MDLM, from time t to time $s < t$. In a single forward pass, the denoiser predicts a probability distribution $\mathbf{p}^l$ over the vocabulary for every masked position, while visible tokens are simply copied. Each masked token is then either unmasked, committing to a single token drawn from its distribution, or left as [MASK]. In the second case, the predicted distribution is discarded, and the next step starts again from the plain [MASK] token.

propagating to subsequent denoising steps. In the example of Figure 2.6, the model is almost undecided between *sat* and *lay* for the third position: this knowledge would be useful context for the next step, but it is completely lost as soon as the position remains masked.

2.6.3 Soft-Masked Diffusion Models

To address the information loss inherent in standard binary unmasking, the Soft-Masked Diffusion Model was proposed by Hersche et al.[5]. Soft-Masking (SM) relaxes this binary constraint by enriching the retained mask representations with a continuous feedback signal—that is, passing the internal predictions of the previous step back into the model as a confidence-weighted superposition of the top- k candidate tokens, rather than resetting to a blank mask.

This continuous feedback mechanism provides significant architectural and performance advantages:

- **Context Preservation Across Steps:** By propagating candidate token distributions instead of hard mask states, partial contextual information can flow across multiple denoising iterations.
- **Dynamic Uncertainty Guidance:** An adaptive, entropy-based weighting mechanism (λ) scales the feedback, preserving the original mask embedding when uncertainty is high while heavily leveraging top predictions when confidence is high.
- **Output Quality:** By enriching the prior at each step, SM enables more accurate sampling, achieving superior output quality compared to binary baselines.

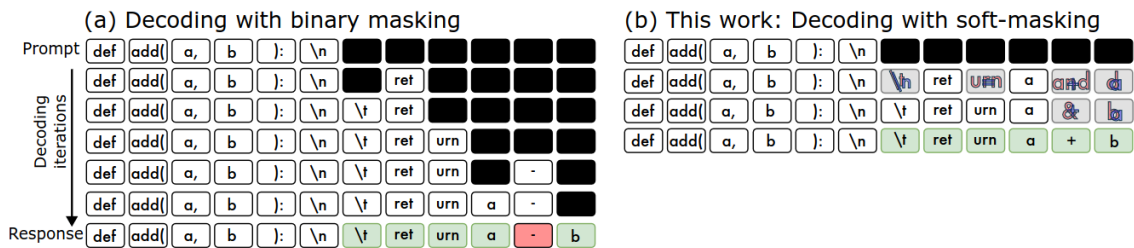


Figure 2.7: Comparison between binary masking (a) and soft masking (b). Soft masking enhances generative quality by denoising masked tokens through previously predicted top- k tokens. Image from [5]

As said before, instead of discarding the output distribution when a token remains masked, SM replaces the pure [MASK] embedding with a convex combination of the mask token and a superposition of the top- k predicted tokens:

$$\mathbf{x}_{t-1}^l = (1 - \lambda(\mathbf{p}_{t-1}^l)) \cdot \mathbf{m} + \lambda(\mathbf{p}_{t-1}^l) \sum_{i \in \text{top-k}(\mathbf{p}_{t-1}^l)} \pi_i \cdot \mathbf{v}_i$$

where:

- $\mathbf{p}_{t-1}^l$ is the probability distribution vector over the entire vocabulary $|V|$ generated by the network at denoising step $t - 1$ for the specific sequence position l ;
- $\mathbf{m}$ is the one-hot vector of the [MASK] token;
- $\mathbf{v}_i$ is a one-hot vector representing token i ;
- π_i is the normalized probability of the top- k tokens:
 $\pi_i = [\mathbf{p}_{t-1}^l]_i / \sum_{j \in \text{top-k}(\mathbf{p}_{t-1}^l)} [\mathbf{p}_{t-1}^l]_j$;
- $\lambda \in [0, 1)$ is the confidence-based weight function.

Soft-Masking is applied exclusively to the [MASK] token. While it is applied with a certain probability during training, it is always enabled during inference.

Confidence-based Weighting

To dynamically balance the contribution of the model's predictions during the reverse process, a confidence-weighted masking strategy is adopted. Formally, the dynamic weight $\lambda(\mathbf{p}_{t-1})$ is computed as follows:

$$\lambda(\mathbf{p}_{t-1}) = \omega_s \cdot \sigma(\omega_a(-H(\mathbf{p}_{t-1}^l) - \omega_b))$$

where $\sigma(\cdot)$ denotes the standard sigmoid activation function, $H(\mathbf{p}_{t-1}^l)$ represents the categorical entropy of the predicted probability distribution over the target tokens at step $t - 1$, and the parameters ω_a , ω_b , ω_s are the steepness ($\omega_a \geq 0$), offset ($\omega_b \leq 0$) and amplitude ($\omega_s \in [0, 1]$) respectively.

The scalar $\lambda(\mathbf{p}_{t-1})$, controls how much of the mask token is replaced by the top- k predicted tokens. A low entropy $H(\mathbf{p}_{t-1}^l)$ signals high prediction confidence, favoring the predicted tokens over the mask embedding. Conversely, a high entropy indicates low confidence, driving λ toward 0 and preserving the mask embedding almost unchanged.

Evaluating this confidence weighting mechanism requires access to the distribution $\mathbf{p}_{t-1}^l$ before performing the actual backward step. Consequently, an additional auxiliary forward pass must be executed at each step t without tracking gradients. This preliminary forward pass allows the model to estimate $H(\mathbf{p}_{t-1}^l)$ and derive $\lambda(\mathbf{p}_{t-1})$ efficiently, which is then used to weight the loss or update step without corrupting the main computation graph.

Chapter 3

Diffusion Language Models in the Reservoir World

In this chapter, we present the architecture and operational principles of **DiESN** (*Diffusion Echo State Network*), the novel sequence modeling architecture developed in this thesis. Specifically, we eliminate the computationally demanding, quadratic-complexity ($\mathcal{O}(L^2)$) self-attention mechanism by replacing it with fixed, non-trainable recurrent layers inspired by Echo State Networks, instantiated through the **ParalESN** formulation. By combining these untrained diagonal recurrent reservoirs with lightweight local depthwise convolutions, modern adaptive normalizations, and an associative scan parallelization, DiESN achieves comprehensive bidirectional sequence mixing with sub-quadratic temporal scaling and a substantially reduced trainable parameter footprint.

3.1 Model Architecture

The overall organization of the DiESN architecture is illustrated in Figure 3.1. The model processes continuous input sequences generated by the diffusion corruption process and is conditioned at every level on the current noise level (timestep).

A continuous sequence of soft-masked token representations and the scalar diffusion noise level σ_t are first mapped into the latent space by the token embedding and timestep embedder modules, respectively. The resulting sequence of embeddings $\mathbf{u}^{(0)}$ is processed through a deep stack of N identical DiESN blocks. Each block carries out global bidirectional

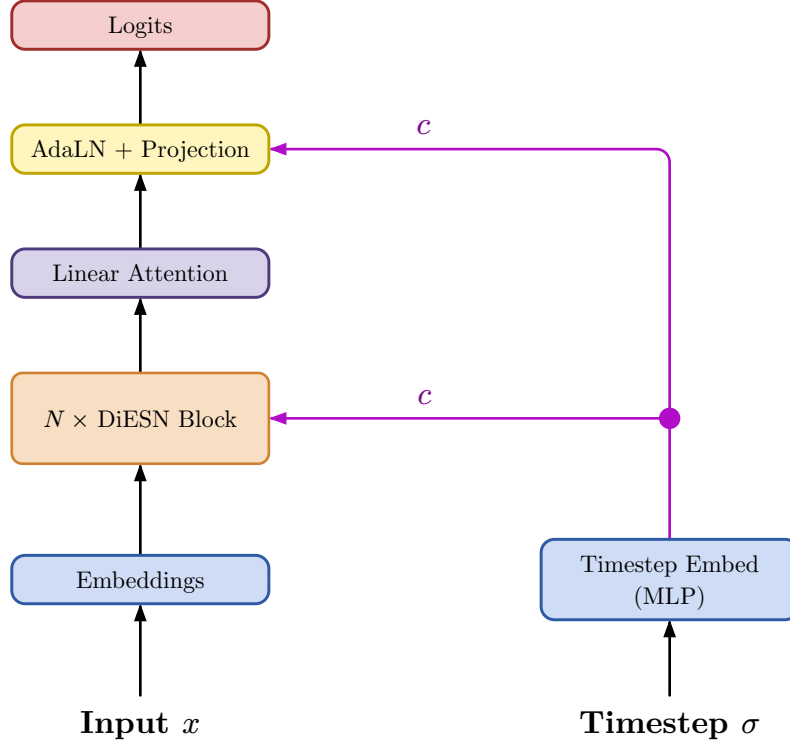


Figure 3.1: Overall architecture of the DiESN architecture. The continuous soft-masked input distributions $\mathbf{x}_t$ and diffusion noise level σ_t are mapped by the token embeddings and timestep embedder into representations $\mathbf{u}^{(0)}$ and conditioning vector $\mathbf{c}$. The representations flow through a stack of N DiESN blocks, a global Flash Linear Attention layer, and an AdaLN-modulated output projection to yield vocabulary logits.

sequence mixing and local feature refinement conditioned on the timestep vector $\mathbf{c}$.

Following the N reservoir-based blocks, a single Flash Linear Attention layer performs a global cross-positional refinement step, after which an adaptive-layer-normalized linear readout head projects the hidden states to unnormalized vocabulary logits.

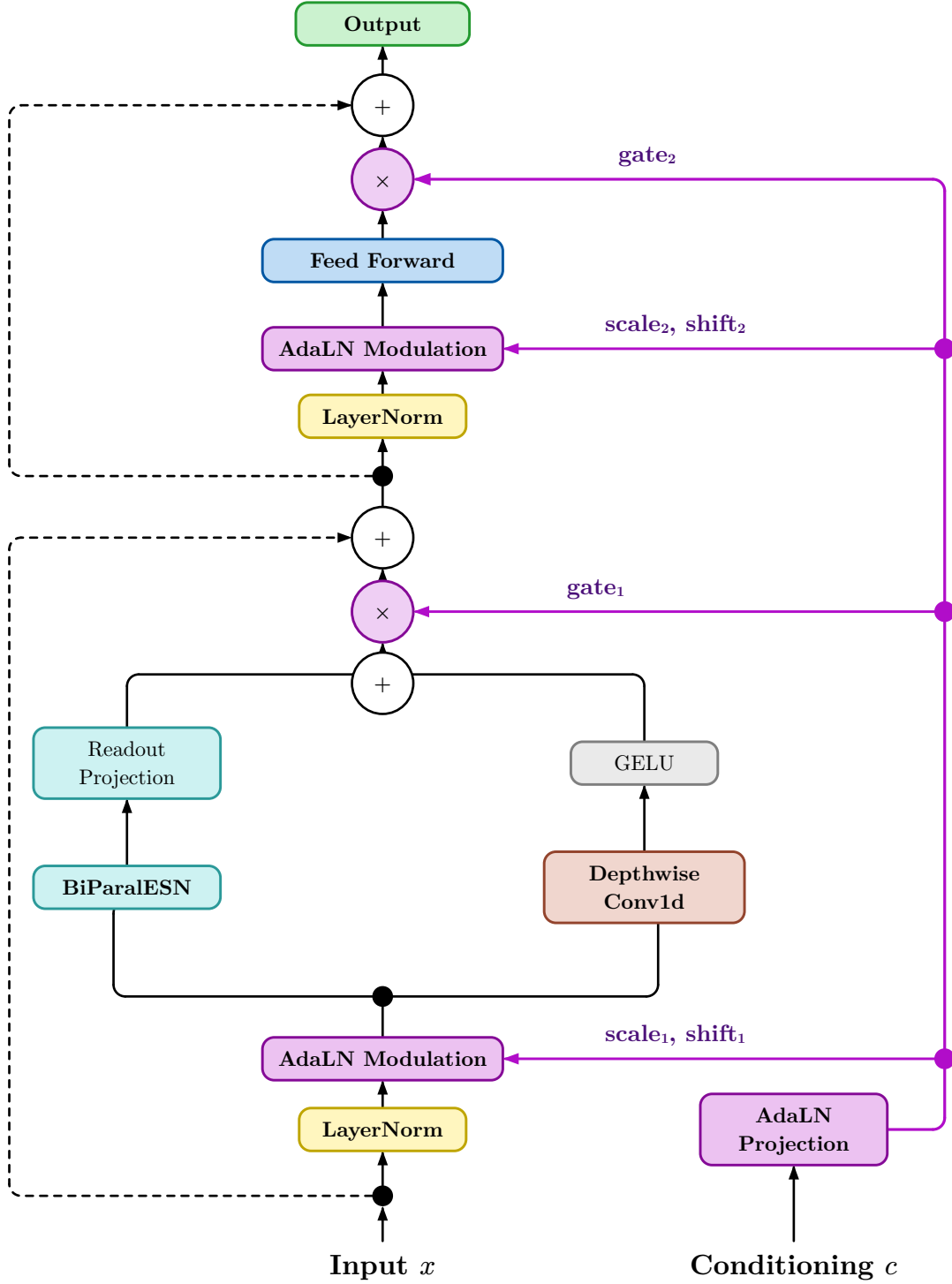


Figure 3.2: Detailed internal schematic of the DiESN Block. The conditioning vector c modulates both sub-layers via an AdaLN-Zero linear projection producing scale, shift, and gate vectors. The first sub-layer performs bidirectional sequence mixing through parallel global (BiParalESN) and local (depthwise Conv1D) branches, fused additively and applied through a gated residual connection with dropout. The second sub-layer comprises a standard position-wise feed-forward network with GELU activation and gated residual addition.

Notation

Throughout this chapter, we index sequence positions by $i \in \{1, \dots, L\}$ and diffusion steps by t ; $m \in \{1, \dots, M\}$ indexes the DiESN blocks. d denotes the model width (hidden dimension), N_h the reservoir size (number of hidden recurrent units), and $\mathcal{V}$ denotes the vocabulary.

Trainable tensors and weight matrices are written in bold roman font ($\mathbf{W}, \mathbf{b}$), whereas frozen, non-trainable reservoir tensors are explicitly designated with an overline ($\overline{\Lambda}, \overline{\mathbf{W}}, \overline{\mathbf{b}}$). Element-wise (Hadamard) vector multiplication is denoted by $\circ$.

3.2 Input and Conditioning

Following the soft-masked diffusion formulation introduced in Section 2 (specifically Section 2.6.3), the input at diffusion step t is a sequence of continuous distributions $\mathbf{x}_t^{1:L}$, with $\mathbf{x}_t^i \in \Delta^{|\mathcal{V}|-1}$ rather than a discrete one-hot vector (only if soft masking is applied).

The embedding layer therefore acts as a continuous weighted lookup:

$$\mathbf{u}_i^{(0)} = \mathbf{E}^\top \mathbf{x}_t^i \in \mathbb{R}^d, \quad \mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d},$$

which smoothly reduces to standard embedding lookup whenever $\mathbf{x}_t^i$ is one-hot (i.e., for completely unmasked positions, as well as for masked positions when the soft-masking interpolation parameter $\lambda = 0$).

The noise level σ_t is mapped to a conditioning vector $\mathbf{c}$ by a dedicated timestep embedder:

$$\mathbf{c} = \mathbf{W}_2^{\text{ts}} \varphi(\mathbf{W}_1^{\text{ts}} \text{SinEmb}(\sigma_t)) \in \mathbb{R}^d,$$

where $\varphi = \text{SiLU}$ and $\text{SinEmb}(\cdot)$ represents the canonical sinusoidal frequency encoding. Crucially, $\mathbf{c}$ is a single vector shared across all sequence positions: conditioning enters the network exclusively through feature-wise adaptive modulation, never along the sequence axis. This keeps the effective sequence length unchanged and eliminates the quadratic overhead that would result from prepending conditioning tokens to self-attention contexts.

3.3 The DiESN Block

The core computational unit of the architecture is the **DiESN Block**, depicted in Figure 3.2. Each block is conditioned through Adaptive Layer Normalization with zero initialization (**AdaLN-Zero**, Peebles & Xie, 2023 [37]). A single linear projection produces six modulation vectors from the timestep conditioning representation $\mathbf{c}$:

$$[\gamma_1, \beta_1, \alpha_1, \gamma_2, \beta_2, \alpha_2] = \mathbf{W}_{\text{ada}}^{(m)} \text{SiLU}(\mathbf{c}) + \mathbf{b}_{\text{ada}}^{(m)} \in \mathbb{R}^{6d},$$

where (γ_1, β_1) and (γ_2, β_2) serve as the scale and shift modulation pairs for the two normalization stages, while $\alpha_1, \alpha_2 \in \mathbb{R}^d$ act as residual gating vectors. The projection weights $\mathbf{W}_{\text{ada}}^{(m)}$ and biases $\mathbf{b}_{\text{ada}}^{(m)}$ are initialized to zero, ensuring that each DiESN block behaves as an identity transformation at the beginning of training, which significantly enhances numerical stability and gradient propagation in deep stacks.

3.3.1 Mixing Stage

Let $\mathbf{u}^{(m-1)} \in \mathbb{R}^{L \times d}$ denote the input to block m . The first sub-layer begins with adaptive pre-normalization:

$$\mathbf{v}_i = \text{LN}\left(\mathbf{u}_i^{(m-1)}\right) \circ (\mathbf{1} + \gamma_1) + \beta_1.$$

The modulated representation $\mathbf{v}$ is subsequently processed by two parallel complementary branches:

1. **Global branch (Bidirectional Linear Reservoir):** To capture long-range contextual dependencies across the entire sequence, the recurrence of ParalESN (see Section 2.2.2) is executed independently in both forward and backward directions:

$$\begin{aligned} \vec{\mathbf{h}}_i &= \overline{\Lambda}_{\rightarrow}^{(m)} \vec{\mathbf{h}}_{i-1} + \tau \left(\overline{\mathbf{W}}_{\rightarrow}^{(m)} \mathbf{v}_i + \overline{\mathbf{b}}_{\rightarrow}^{(m)} \right), & \vec{\mathbf{h}}_0 &= \mathbf{0}, \\ \tilde{\mathbf{h}}_i &= \overline{\Lambda}_{\leftarrow}^{(m)} \tilde{\mathbf{h}}_{i+1} + \tau \left(\overline{\mathbf{W}}_{\leftarrow}^{(m)} \mathbf{v}_i + \overline{\mathbf{b}}_{\leftarrow}^{(m)} \right), & \tilde{\mathbf{h}}_{L+1} &= \mathbf{0}, \end{aligned}$$

where $\overline{\Lambda}_{\rightarrow}^{(m)} = (1 - \tau)I + \tau\Lambda_{\rightarrow}^{(m)}$ and $\overline{\Lambda}_{\leftarrow}^{(m)} = (1 - \tau)I + \tau\Lambda_{\leftarrow}^{(m)}$ denote the effective diagonal transition matrices incorporating the leaky integration rate $\tau \in (0, 1]$, in accordance with the ParalESN formulation (Section 2.2.2).

$\vec{h}_i, \tilde{h}_i \in \mathbb{C}^{N_h}$ represent the bidirectionality (forward and backward pass) of the model. Both directions are evaluated in parallel using the associative scan algorithm in $\mathcal{O}(\log L)$ parallel depth (see Section 2.2.2).

Bidirectionality is fundamentally indispensable in this setting: unlike autoregressive language modeling, where causal masking restricts attention to past tokens, text diffusion denoising is an inherently non-causal problem.

The complex reservoir states from both directions are concatenated into a real-valued representation and projected back to the model dimension:

$$\mathbf{h}_i = [\Re(\vec{h}_i); \Im(\vec{h}_i); \Re(\tilde{h}_i); \Im(\tilde{h}_i)] \in \mathbb{R}^{4N_h},$$

$$\mathbf{g}_i = \mathbf{W}_{\text{proj}}^{(m)} \mathbf{h}_i,$$

where $\mathbf{W}_{\text{proj}}^{(m)} \in \mathbb{R}^{d \times 4N_h}$ is a fixed projection matrix initialized using Kaiming uniform initialization [38] and without bias. This matrix primarily serves to reduce the double dimensionality to d , resulting from the bidirectionality of ParalESN.

2. **Local branch (Depthwise Convolution):** In parallel with the reservoir, a local branch applies a depthwise 1-D convolution along the sequence axis with kernel size k_c , followed by a GELU [39] non-linearity:

$$\ell_i = \varphi_{\text{gelu}} \left(\left(\mathbf{W}_{\text{conv}}^{(m)} * \mathbf{v} \right)_i + \mathbf{b}_{\text{conv}}^{(m)} \right).$$

This local branch captures and enriches the local context through interactions among adjacent tokens. Additionally, it constitutes the sole source of sequence-wise non-linearity in the mixing stage.

The global and local representations are combined additively and passed through a gated residual connection with dropout:

$$\mathbf{z}_i^{(1)} = \mathbf{g}_i + \ell_i,$$

$$\mathbf{y}_i = \mathbf{u}_i^{(m-1)} + \alpha_1 \circ \text{Drop}(\mathbf{z}_i^{(1)}).$$

3.3.2 Feed-Forward Stage

The second half of the DiESN block comprises a position-wise Multi-Layer Perceptron (FFN) modulated by the second set of AdaLN parameters:

$$\begin{aligned} \mathbf{w}_i &= \text{LN}(\mathbf{y}_i) \circ (\mathbf{1} + \gamma_2) + \beta_2, \\ \mathbf{z}_i^{(2)} &= \mathbf{W}_2^{(m)} \varphi_{\text{gelu}} \left(\mathbf{W}_1^{(m)} \mathbf{w}_i + \mathbf{b}_1^{(m)} \right) + \mathbf{b}_2^{(m)}, \\ \mathbf{u}_i^{(m)} &= \mathbf{y}_i + \alpha_2 \circ \text{Drop} \left(\mathbf{z}_i^{(2)} \right), \end{aligned}$$

where $\mathbf{W}_1^{(m)} \in \mathbb{R}^{d_{\text{ff}} \times d}$ and $\mathbf{W}_2^{(m)} \in \mathbb{R}^{d \times d_{\text{ff}}}$, with the intermediate hidden expansion dimension set to $d_{\text{ff}} = 4d$. The gated residual connection scales the FFN update by α_2 , completing the block transformation. The MLP acts as our Readout as described in Section 2.2.2. However, since the readout layer is optimized iteratively using backpropagation, closed-form methods (such as Ridge Regression) cannot be directly applied.

3.4 Output Head

Following the stack of M DiESN blocks, the sequence representation $\mathbf{u}^{(M)}$ is refined globally before readout. A single Flash Linear Attention (Gated Linear Attention, GLA) [18] layer mixes representations globally across the sequence:

$$\tilde{\mathbf{u}} = \text{GLA}(\mathbf{u}^{(M)}).$$

The output head then applies a final AdaLN modulation followed by an untied linear projection to the vocabulary space to produce the logits:

$$[\gamma_f, \beta_f] = \mathbf{W}_{\text{ada}}^f \text{SiLU}(\mathbf{c}) + \mathbf{b}_{\text{ada}}^f,$$

$$\ell_i^{\text{logits}} = \mathbf{W}_{\text{out}} (\text{LN}(\tilde{\mathbf{u}}_i) \circ (\mathbf{1} + \gamma_f) + \beta_f) + \mathbf{b}_{\text{out}}, \quad \mathbf{W}_{\text{out}} \in \mathbb{R}^{|\mathcal{V}| \times d}.$$

The unnormalized logits ℓ^{logits} are converted into predicted categorical probability distributions $\mathbf{p}_{t-1}^{1:L}$ by a softmax operation. These probabilities are then routed to the unmasking and soft-masking modules described in Section 2 (Section 2.6.2 and Section 2.6.3), determining the sequence state for step $t - 1$.

3.5 Trainable and Frozen Parameters

A fundamental advantage of the DiESN architecture lies in its partitioning between trainable and frozen parameters, inheriting the training efficiency of the Reservoir Computing paradigm.

The set of frozen parameters is given by:

$$\mathcal{P}_{\text{frozen}} = \left\{ \overline{\Lambda}_{\rightarrow}^{(m)}, \overline{\Lambda}_{\leftarrow}^{(m)}, \overline{\mathbf{W}}_{\rightarrow}^{(m)}, \overline{\mathbf{W}}_{\leftarrow}^{(m)}, \overline{\mathbf{b}}_{\rightarrow}^{(m)}, \overline{\mathbf{b}}_{\leftarrow}^{(m)}, \overline{\mathbf{W}}_{\text{proj}} \right\}_{m=1}^M.$$

These recurrent transition matrices and input projections are sampled once at initialization in accordance with the ParalESN specification (see Section 2.2.2) and remain completely fixed throughout all training iterations.

Because no gradients are accumulated for the recurrent parameters themselves, the backward pass never calculates the gradient nor does the optimizer allocate momentum and variance buffers for them.

Furthermore, because the spectral condition $|\overline{\lambda}_j| < 1$ is enforced strictly at initialization and never perturbed by gradient updates, the **ESP** holds unconditionally throughout the entire training process. Consequently, DiESN guarantees internal dynamical stability without requiring eigenvalue clipping, spectral re-parameterization, or complex constrained optimization schemes.

3.6 Training Objective

The DiESN architecture is trained by minimizing the Negative Log-Likelihood (NLL) objective:

$$\mathcal{L}(\theta) = -\mathbb{E}_{t \sim U(0,1), \mathbf{x}_0 \sim q_{\text{data}}(\cdot), \mathbf{x}_t \sim q(\cdot | \mathbf{x}_0)} \left[\frac{1}{t} \sum_{i=1}^L \mathbf{1}_{\mathbf{x}_t^i = m} \log \left((\mathbf{x}_0^i)^\top g_\theta(\mathbf{x}_0^i | \mathbf{x}_t^{1:L}) \right) \right]$$

The factor $1/t$ acts as the per-token normalization of the loss: since about $t * L$ tokens are masked at noise level t , it makes the contribution of the masked tokens comparable across noise levels.

Chapter 4

Experiments

In this chapter, we evaluate the performance and efficiency of incorporating the Parallel Echo State Network (ParalESN) architecture as the backbone within a Soft-Masking Diffusion Language Model. The primary objective is to demonstrate that replacing the traditional Self-Attention mechanism (Transformer) with ParalESN yields comparable generative capabilities while affording significant advantages in computational efficiency, memory consumption, and long-context scalability.

We compare ParalESN against state-of-the-art fully trainable sequence models, namely the standard Transformer, MAMBA, and S5 as backbones of the diffusion model framework. The evaluation spans synthetic datasets, real-world text corpora, and targeted benchmarks explicitly designed to stress-test long-term memory retention and hardware utilization.

In all the experiments the soft-masking is always enabled with probability $p = 0.8$

4.1 ParalESN Configurations

To adapt the ParalESN backbone to the Soft-Masking diffusion framework, we identified two optimal hyperparameter configurations through Bayesian optimization [40]. Configuration 1 was optimized on the TinyStories dataset, characterizing a short-context setting, while Configuration 2 was optimized on the first 20 million tokens of OpenWebText, addressing longer contexts.

Parameter	Conf 1 (Short Context)	Conf 2 (Long Context)
n_layers	1	1
bidirectional	true	true
bid_merge	“cat”	“cat”
leaky	0.9	0.9
in_initialization	“gamma”	“gamma”
in_scaling	0.3668	0.3668
rho_min	0.4	0.4
rho_max	0.8	0.9
phase_min	1.57	0.0
phase_max	1.57	0.1963
esn_kernel_size	3	3
esn_mix_scaling	0.1	0.1

Table 4.1: Comparison of ParalESN Configurations.

A key distinction between the two configurations lies in the phase initialization constraints. In Configuration 1, both minimum and maximum phases are fixed to 1.57 ($\approx \frac{\pi}{2}$). In the ParalESN model, the recurrent transition matrix is diagonal and composed of eigenvalues defined as $\lambda = \rho e^{i\theta}$. When the phase is fixed at $\theta = \frac{\pi}{2}$, the complex exponential becomes:

$$e^{i\frac{\pi}{2}} = \cos\left(\frac{\pi}{2}\right) + i \sin\left(\frac{\pi}{2}\right) = 0 + i = i$$

So the eigenvalues of the transition matrix have zero real part ($\lambda = i\rho$). Considering the case with leaky rate $\tau = 1$ and no external input, at each time step a 90° rotation occurs in the complex plane:

$$h_t \approx \lambda h_{t-1} = (i\rho)(a_{t-1} + ib_{t-1}) = -\rho b_{t-1} + i(\rho a_{t-1})$$

As shown by the equation, the real and imaginary parts swap channels at every step (with a sign reversal on the real part), preventing the signal from accumulating monotonically along the same direction. In the absence of external inputs, the trajectory of each channel completes a full orthogonal cycle every $T = \frac{2\pi}{\theta} = \frac{2\pi}{1.57} = 4$ steps, making this configuration ideal for isolating local short-range correlations and contrasts.

Conversely, Configuration 2, tailored for longer texts, sets a phase range near zero. This forces the imaginary components of the eigenvalues to remain

close to zero while making the real part dominant ($\cos(\theta) \approx 1, \sin(\theta) \approx 0$). Assuming $\tau = 1$ and omitting the input term, the recurrence is described by:

$$h_t \approx \lambda h_{t-1} = \rho(\cos(\theta) + i \sin(\theta))(a_{t-1} + ib_{t-1})$$

Separating the real and imaginary parts:

$$\begin{aligned}\Re(h_t) &\approx \rho(a_{t-1} \cos(\theta) - b_{t-1} \sin(\theta)) \\ \text{Im}(h_t) &\approx \rho(a_{t-1} \sin(\theta) + b_{t-1} \cos(\theta))\end{aligned}$$

In this case, there is no channel switching, and information accumulates over time, exhibiting an oscillatory dynamic greater than 4 ($T \gg 4$).

4.2 Evaluation

For all experiments, model quality is quantified using Perplexity (PPL). As described in Section 3.6, the loss $\mathcal{L}$ is already normalized per token through the factor $1/t$, so the perplexity is simply its exponential, evaluated on the validation set:

$$\text{PPL} \leq \exp(\mathcal{L}_{\text{val}}(\theta))$$

A lower PPL indicates a more accurate probability distribution over the vocabulary. However, since in discrete Diffusion Models the marginal likelihood $p_\theta(\mathbf{x}_0)$ is intractable, $\mathcal{L}$ is an upper bound (NELBO) on the true negative log-likelihood, and the reported PPL is therefore an upper bound on the true perplexity.

Due to the high computational cost of pre-training and large-scale benchmarking, all reported results are derived from a single run per configuration (unless otherwise specified). In all tables, the best result will be indicated in bold, and the second-best result will be underlined.

The datasets and tasks used in the experiments are summarized in Table 4.2. They are grouped into three categories: synthetic text corpora, used to test the models in a controlled setting; real-world text corpora, used to assess language modeling on natural and more diverse text; and targeted benchmarks, designed to isolate specific properties such as memory retrieval and hierarchical reasoning.

Dataset	Type	Tokens	Tokenizer	Context Length	Metric
<i>Synthetic text corpora</i>					
TinyStories	LLM-generated short stories	$\approx 460\text{M}$	BERT	20,000	PPL
TextBooks	LLM-generated textbooks	$\approx 439\text{M}$	BERT	20,000	PPL
<i>Real-world text corpora</i>					
Wikipedia	Encyclopedic articles	$\approx 2\text{B}$	BERT	8192	PPL
OpenWebText	Web pages	$\approx 9\text{B}$ tokens	GPT-2	1024	PPL
<i>Targeted benchmarks</i>					
Sequence Copy	Memory retrieval	Randomly generated	26 symbols + 4 special tokens	train: $L \in [2, 15]$ test: $L = 10, 20, 30$	Token accuracy
Dyck-4	Hierarchical reasoning	Generated by a PCFG	4 bracket pairs	256, 1024, 4096	Accuracy

Table 4.2: Overview of the datasets and tasks used in the experiments. For the Sequence Copy task, L denotes the length of the sequence to be copied. PCFG stands for “Probabilistic Context-Free Grammar” since data generation is based on the parentheses.

4.3 Synthetic Datasets

We first evaluate the models on synthetic datasets, which provide controlled environments with constrained vocabularies and simplified linguistic structures.

4.3.1 TinyStories

TinyStories [41] is a synthetic dataset generated using an LLM, comprising short stories with a simplistic vocabulary tailored for a 3-to-4-year-old reading level. The dataset encompasses approximately 460 million tokens. For this experiment, we use Configuration 1 of Table 4.1, as the narrative structure fundamentally relies on short-range dependencies.

In this setting, multiple stories are concatenated within the same input sequence, delimited by the special beginning-of-sequence [BOS] and end-

of-sequence [EOS] tokens. This approach is designed to demonstrate the linear complexity of ParalESN compared to the quadratic computational cost typical of standard Transformers. Notably, ParalESN selectively mimics the properties of attention, successfully isolating individual sequences.

Parameter	Value
Model Parameters	$\approx 30\text{M}$
Tokenizer	BERT
Context Length	20,000
Batch Size	1
Learning Rate	3×10^{-4}
Epochs	1

Table 4.3: Training Specifications on TinyStories

Results

The model was trained using the standard train/validation split provided by Hugging Face [42]. Throughout training, the model processed ≈ 420 million tokens. Model selection was performed based on the minimum validation loss.

Model	PPL	Time	Train. Params	Fixed Params	Total Params
GRU	–	3d 18h 08m	29.6 M	–	29.6 M
LRU	11.47	9h 47m 06s	29.6 M	–	29.6 M
Reservoir	13.07	13h 56m 42s	23.2 M	3.2 M	26.4 M
S5	<u>10.80</u>	<u>1h 27m 02s</u>	25.6 M	–	25.6 M
MAMBA	8.76	6h 04m 31s	22.7 M	–	22.7 M
Transformer	<u>10.80</u>	2h 10m 07s	27.6 M	–	27.6 M
ParalESN	11.82	1h 22m 09s	20.1 M	3.9 M	24.0 M

Table 4.4: Performance and efficiency comparison on TinyStories.

While MAMBA achieves the most competitive Perplexity, ParalESN establishes itself as the most efficient architecture, minimizing the training time to approximately 1.3 hours. It yields a PPL broadly comparable to that of the Transformer and S5 but trains significantly faster. Although its Perplexity does not match the peak score set by MAMBA—reflecting the inherent

trade-off of using a fixed, untrained dynamical core—ParalESN achieves an optimal balance between competitive performance and resource efficiency.

Based on these results, purely sequential RNNs (GRU, LRU, standard Reservoir) are excluded from subsequent tests, retaining only ParalESN, Transformer, MAMBA, and S5 to ensure fair hardware utilization comparisons.

4.3.2 TextBooks

The TextBooks dataset [43] represents a natural evolution of TinyStories. It comprises 650,000 unique synthetic textbooks containing approximately 439 million tokens. Characterized by longer continuous texts, this benchmark requires Configuration 2 of Table 4.1.

Parameter	Value
Model Parameters	$\approx 23\text{M}$
Tokenizer	BERT
Context Length	20,000
Batch Size	1
Learning Rate	3×10^{-4}
Epochs	1

Table 4.5: Training Specifications on TextBooks

Results

The model was trained using the standard train split provided by Hugging Face [44] and using the last 10,000 documents of the split as validation set. Model selection was performed based on the minimum validation loss.

Model	PPL	Time	Train. Params	Fixed Params	Total Params
S5	12.18	1h 07m 20s	23.3 M	–	23.3 M
MAMBA	9.03	1h 57m 45s	19.6 M	–	19.6 M
Transformer	11.59	1h 19m 06s	21.7 M	–	21.7 M
ParalESN	<u>10.38</u>	<u>1h 15m 38s</u>	21.7 M	7.09 M	28.79 M

Table 4.6: Performance comparison across architectures on TextBooks.

Consistent with the TinyStories results, MAMBA retains the best PPL result, likely due to its dynamic sequence-selectivity mechanism, albeit at the cost of a longer training time. ParaLESN emerges as highly competitive, marginally outperforming the Transformer in both training speed and validation Perplexity.

4.4 Real-World Datasets

Transitioning from synthetic data, we benchmark the models on unstructured, real-world text corpora to assess their capacity to model complex, natural linguistic distributions.

4.4.1 Wikipedia

We utilize a cleaned subset of the English Wikipedia dump [45], constrained to ≈ 2 billion tokens (40% of the original dataset). The primary objective is to verify whether ParaLESN can rival the Transformer’s expressiveness under medium context lengths (8,192 tokens) on a highly diverse vocabulary. Also this benchmark requires Configuration 2 of Table 4.1.

Parameter	Value
Model Parameters	$\approx 60\text{M}$
Tokenizer	BERT
Context Length	8192
Batch Size	1
Learning Rate	$3 \times 10^{\{-4\}}$
Epochs	2

Table 4.7: Training Specifications on Wikipedia

Training was extended to two epochs, as empirical observations indicated that the first epoch primarily served as structural alignment given the dataset’s vast linguistic diversity.

Results

The model was trained using the ‘20231101.en’ split provided by Hugging Face [46] and using the last 10,000 documents of the split as validation set. Model selection was performed based on the minimum validation loss.

Model	PPL	Time	Train. Params	Fixed Params	Total Params
S5	51.07	11h 20m 38s	53.8 M	–	53.8 M
MAMBA	42.95	15h 32m 06s	50.9 M	–	50.9 M
Transformer	<u>49.90</u>	<u>11h 19m 34s</u>	49.8 M	–	49.8 M
ParalESN	50.19	11h 09m 24s	42.9 M	10.0 M	52.9 M

Table 4.8: Performance comparison across architectures on Wikipedia.

While MAMBA achieves the lowest overall perplexity, its significantly higher execution time contrasts with ParalESN, which achieves the fastest training time, although only by a few minutes, while maintaining a competitive PPL. This establishes ParalESN as the best overall trade-off between predictive performance and training speed. Nevertheless, the overall performance gap across architectures remains moderate: ParalESN performs on par with the standard Transformer, despite keeping 10,0M of its parameters fixed.

4.4.2 Preliminary Transfer Experiment on OpenWebText

OpenWebText [47] is an open-source replication of the WebText dataset utilized for GPT-2 [48], constructed by extracting HTML content from highly upvoted Reddit URLs. We utilized a subset of approximately 9 billion English tokens.

Due to hardware limitations, we adopted a transfer learning paradigm. We initialized our architectures using a pre-trained Transformer checkpoint from the original Soft-Masking paper. Specifically, the Embedding matrix, Output Layer, and the Multi-Layer Perceptron (MLP) responsible for the Soft-Masking logic were transferred and frozen. The backbones (ParalESN and S5) were subsequently trained from scratch to bridge the latent representations. MAMBA was excluded from this experiment due to prohibitively long training estimations. Also this benchmark requires Configuration 2 of Table 4.1.

Parameter	Value
Model Parameters	$\approx 179\text{M}$
Tokenizer	GPT-2
Context Length	1024
Local Batch Size	32
Grad. Accumulation	16
Global Batch Size	2048
Learning Rate	3×10^{-4}
Epochs	2
GPUs	4

Table 4.9: Training Specifications on OpenWebText

Results

The model was trained using the standard train split provided by Hugging Face [49] and using the last 100,000 documents of the split as validation set. Throughout training, the model processed $\approx 9\text{B}$ tokens. Model selection was performed based on the minimum validation loss. Training is done on 2 epochs since one epoch was not enough to obtain an optimal PPL.

Model	PPL	Time	Train. Params	Fixed Params	Total Params
S5	70.52	32h 04m 48s	101.5 M	77.5 M	179.0 M
Transformer	64.02	<u>31h 53m 34s</u>	101.5 M	77.5 M	179.0 M
ParalESN	<u>68.03</u>	31h 44m 40s	66.6 M	112.5 M	179.0 M

Table 4.10: Performance comparison across architectures on OpenWebText.

After two training epochs, all architectures successfully exit the initial random state and begin adapting well to the task. However, the perplexity values remain noticeably higher than in previous experiments simply because two epochs are not enough, requiring more training time to fully converge. As expected, the Transformer achieves the lowest perplexity, since the pre-trained weights were originally tailored to its own architecture. Nevertheless, these early results show that ParalESN easily adapts when replacing the Transformer’s Attention mechanism, securing second place ahead of S5, with a training time comparable to that of the other models.

4.4.3 Observed Scaling Challenges in Text Diffusion

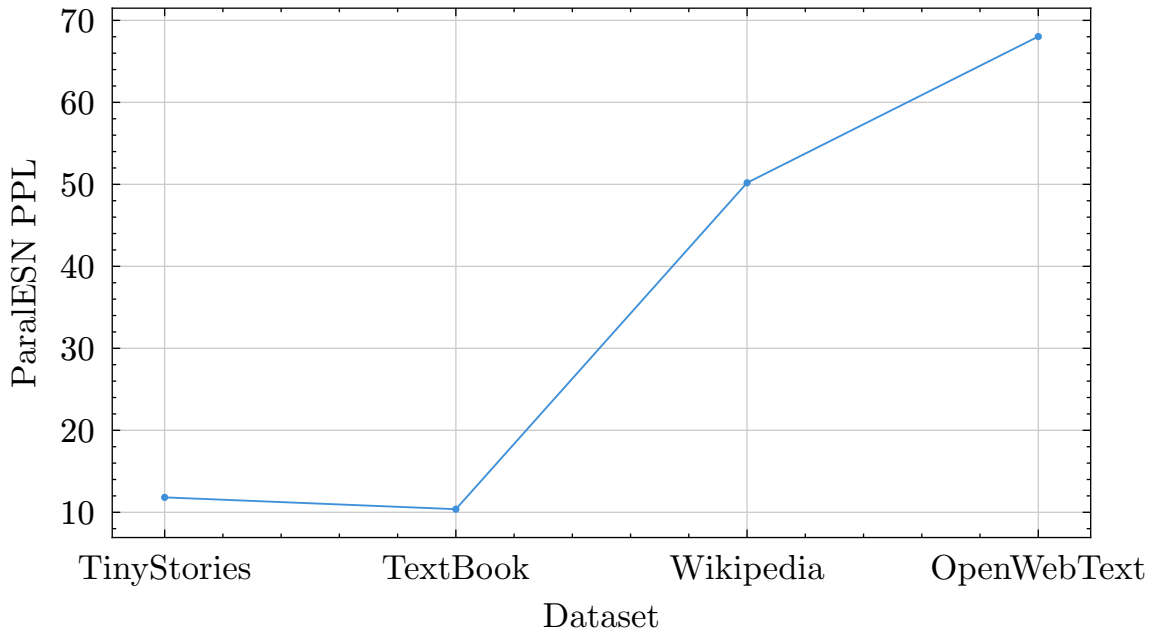


Figure 4.1: ParaESN perplexity measured across the evaluation datasets.

As illustrated in Figure 4.1, it is interesting to observe the trend of perplexity (PPL) in relation to the overall token count. The experimental results suggest that transitioning from synthetic to real-world datasets, combined with an increase in token volume, may lead to higher PPL values or require significantly longer training times to achieve optimal performance. However, these findings must be interpreted with caution. The evaluation setup varies across datasets, as different tokenizers were employed and each corpus inherently exhibits distinct noise characteristics and text complexity. Consequently, while these results point to the scaling challenges of text diffusion models—where larger and more diverse batches lead to more denoising steps—the figure alone cannot determine whether the higher PPL stems strictly from dataset scale or from these confounding experimental parameters.

4.5 Targeted Benchmarks

The following experiments are deliberately constructed to isolate and evaluate specific architectural properties: long-term dependency retrieval, memory consumption, and temporal inference efficiency.

4.5.1 Sequence Copy

The sequence copy task is a standard benchmark for evaluating the memory retrieval capabilities of sequence models. This benchmark was introduced in paper [50] to demonstrate the ability of attention mechanisms, such as Transformers, to attend to important tokens while filtering out noise in the sequences. The task assesses two fundamental properties:

- The ability to **retrieve a specific subsequence** from the preceding context and reproduce it exactly, without distortion or hallucination;
- The capacity to **preserve and retrieve information over long contexts**, by varying either the sequence length or the distance between the source tokens and the position at which they must be regenerated.

Formally, the task is defined as follows. Let $\mathcal{V} = \{v_1, \dots, v_K\}$ denote an alphabet of $K = 26$ information-bearing symbols, and let $\mathcal{S} = \{\text{BOS}, \text{COPY}, \text{EOS}, \text{PAD}\}$ denote a set of special tokens. Given a source sequence $\mathbf{s} = (s_1, \dots, s_L)$ with $s_i \in \mathcal{V}$ sampled uniformly at random, the model receives as input the concatenation

$$x = \langle \text{BOS}, s_1, \dots, s_L, \text{COPY}, \text{PAD}, \dots, \text{PAD}, \text{EOS} \rangle$$

where the L **PAD** positions following the **COPY** trigger constitute the masked target region. The model is trained to predict the ground-truth token s_i at each masked position i , using the cross-entropy loss over the predicted distribution $\hat{y}$ and the one-hot target y :

$$\mathcal{L}(y, \hat{y}) = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

The task therefore requires the model to:

- Recognise the **COPY** token as a retrieval trigger
- Regenerate the input sequence in the correct order at the masked output positions.

During training, sequence lengths are sampled uniformly from the range $L \in [2, 15]$, thereby exposing the model to variable-length inputs and simulating the heterogeneous context lengths encountered in natural language data.

Parameter	Value
Hidden size	256
N. of layers	6
Dropout	0.0
Batch size	64
Training steps	1000

Table 4.11: Common training specifications for the Sequence Copy task. ParalESN uses Configuration 1 from Table 4.1, given the short sequence lengths involved in this benchmark.

Results

For the sequence copy experiment, the convergence plot below illustrates the accuracy progression across steps evaluated on the training set, while the validation results are reported separately in the Table 4.12 using a dedicated validation set. Accuracy is defined as the ratio of correctly predicted tokens divided by the total tokens.

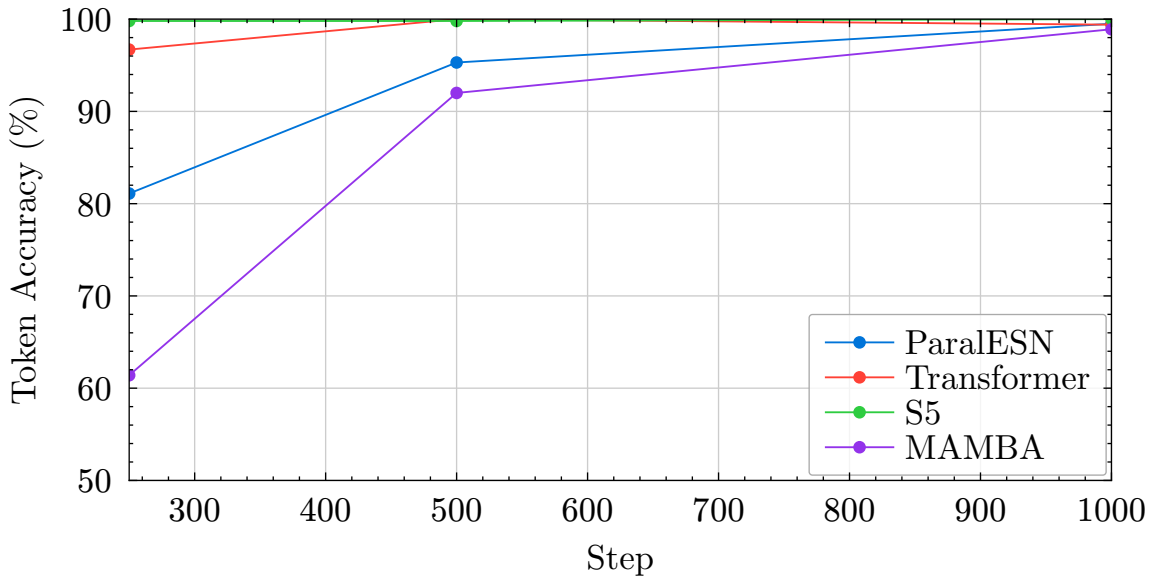


Figure 4.2: Training convergence on the Sequence Copy task on the train set. Token accuracy (%) is reported at 250, 500, and 1,000 training steps. Token accuracy is defined as the fraction of individual tokens in the copied output that exactly match the corresponding tokens in the source sequence.

The evaluation is structured along two axes, designed to assess complementary aspects of model performance:

- **In-Distribution** ($L = 10$): measuring token-level accuracy on sequence lengths seen during training, thereby testing the model’s ability to faithfully memorise and reproduce sequences under familiar conditions.
- **Extrapolation (Out-of-Distribution)** ($L = 20$, $L = 30$): evaluating the capacity for length generalisation on sequences that are longer than the maximum training length, thereby probing whether each architecture’s memory mechanism degrades gracefully or catastrophically beyond the training distribution.

Model	In-Distribution ($L = 10$)	Extrapolation ($L = 20$)	Extrapolation ($L = 30$)
S5	90.9%	44.2%	<u>29.7%</u>
MAMBA	87.3%	<u>46.8%</u>	26.7%
Transformer	99.9%	4.7%	4.3%
ParalESN	<u>99.6%</u>	68.2%	41.1%

Table 4.12: The Transformer and ParalESN achieve near-perfect accuracy on seen length (In-Distribution). Under sequence length extrapolation (Out-of-Distribution), the Transformer suffers a total collapse ($\approx 4\%$), whereas ParalESN exhibits superior generalisation.

The Transformer’s collapse on out-of-distribution lengths is most likely a consequence of RoPE [51], which encodes positional information by rotating query and key vectors by angles proportional to their position. Since training covers only sequences up to $L = 15$, longer inputs produce rotation angles (and relative distances between tokens) that the model has never seen, causing the attention mechanism to assign incorrect weights and disrupting the positional retrieval that copying requires. This limitation is well documented in the literature: Transformers with RoPE are known to generalize poorly to sequences longer than those seen during training [52], [53], and attention scores can become unstable at unseen positions [54].

ParalESN’s superior extrapolation derives from the fact that its state representation is entirely length-agnostic. As discussed in Section 2.2.2, the reservoir state evolves via a recurrence over a diagonal transition matrix with complex eigenvalues. This recurrence contains no positional parameters: the hidden state at any step t is a linear combination of all preceding inputs weighted by powers of the eigenvalues, and this expression is valid for arbitrary t .

The gradual accuracy decline from 99.6% to 68.2% and 41.1% reflects the fading memory property inherent to ESP-satisfying systems. Nevertheless, this remains a remarkable result, especially given that the model was only pre-trained on a synthetic dataset specific to this task. Moreover, this performance is still roughly ten times higher than that of the Transformer, which struggles significantly under the same conditions.

Mamba’s intermediate extrapolation performance (46.8% at $L = 20$, 26.7% at $L = 30$) can be attributed to its selective state space mechanism (Figure 2.2), which makes the recurrence dynamics input-dependent. Unlike ParalESN’s fixed reservoir, Mamba learns to modulate its transition matrices using the current input, introducing a data-dependent filtering during training. When confronted with longer sequences, these learned selective gates encounter input patterns and state configurations outside their training distribution, causing suboptimal state updates that progressively degrade the fidelity of the copied output.

In summary, the sequence copy task exposes a fundamental trade-off: the Transformer achieves near-perfect in-distribution accuracy through direct positional retrieval but fails catastrophically on unseen lengths due to its dependence on seen position encodings, that’s why, currently, additional training is required to achieve broader contexts; Mamba’s input-dependent selectivity partially generalises but remains bound to training-length dynamics; ParalESN, by contrast, relies on a fixed, length-agnostic linear recurrence whose stability is guaranteed by the ESP.

4.5.2 Dyck-4 Language Test

The Dyck- k language task [55] constitutes a rigorous benchmark for assessing a model’s hierarchical reasoning and unbounded memory retention. Mathematically, given a vocabulary of k pairs of matching brackets, a valid sequence is constructed by correctly nesting these brackets. For the Dyck-4 language test, the vocabulary consists of four types of bracket pairs: $()$, $[]$, $\{ \}$, and $< >$. We generate syntactically valid prefixes of length L via a probabilistic Context-Free Grammar. The model’s objective is to accurately predict the closing bracket at position $L + 1$ that maintains the syntactic validity of the sequence. This task is notoriously challenging for standard Transformers, as their attention mechanisms often fail to track deep hierarchical structures over long contexts without positional deterioration.

Parameter	Value
Model Parameters	$\approx 1\text{M}$
Context Length (L)	256, 1024, 4096
Batch Size	128
Learning Rate	$3 \times 10^{\{-4\}}$
Training Steps	10,000

Table 4.13: Training Specifications for Dyck-4 Test

Results

Model	Accuracy ($L = 256$)	Accuracy ($L = 1024$)	Accuracy ($L = 4096$)
S5	<u>86.47%</u>	<u>88.59%</u>	88.27%
MAMBA	89.80%	90.64%	90.96%
Transformer	66.36%	55.50%	34.70%
ParalESN	85.48%	87.18%	<u>86.82%</u>

Table 4.14: Dyck-4 Accuracy variation with respect to context length L .

As theorized, the accuracy of the Transformer drops from 66% to 34% as the sequence length escalates to 4,096. Conversely, state-space baselines such as Mamba and S5 reach slightly higher absolute accuracy scores. However ParalESN demonstrates remarkable structural stability and robustness across increasing context window lengths, maintaining a constant accuracy of roughly 87% all the way from 256 to 4,096 tokens.

4.5.3 Computational Efficiency on Long Contexts

To quantify computational overhead, we plot the execution times (average forward and backward pass duration) against context lengths up to 22,528 tokens.

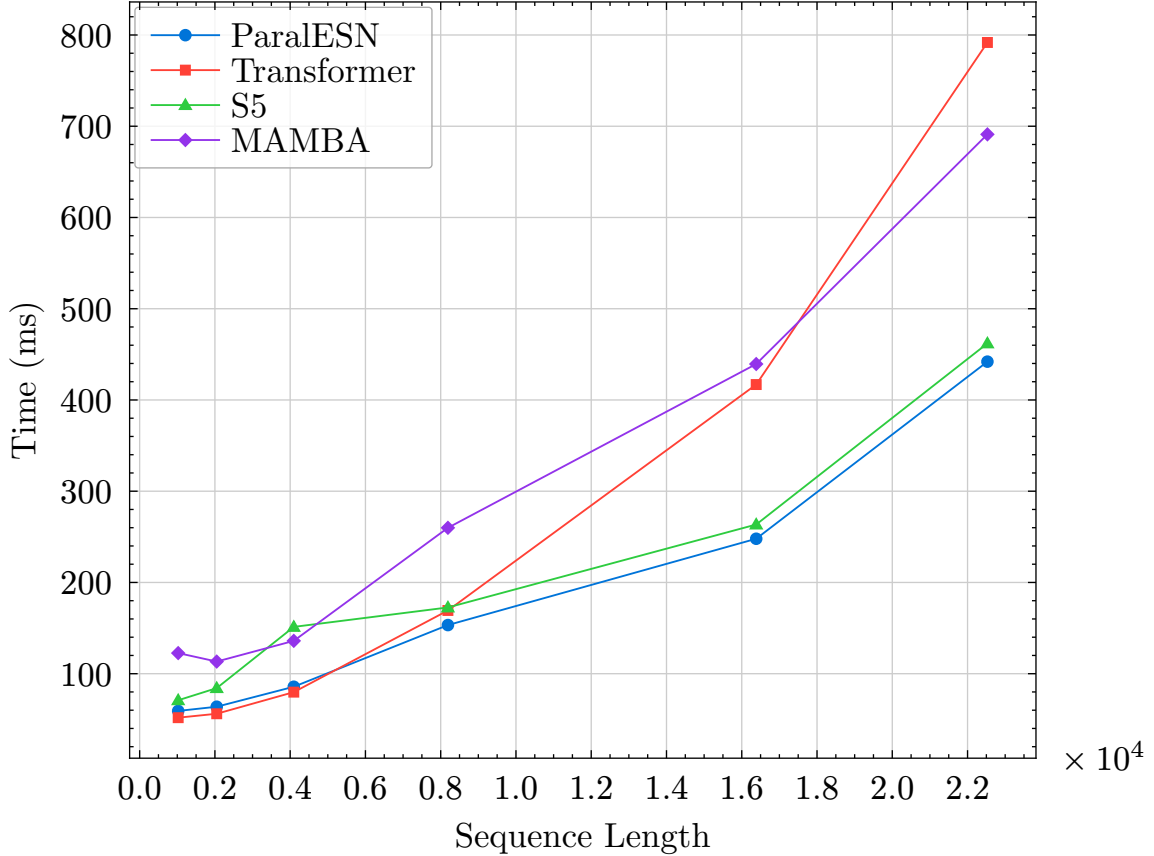


Figure 4.3: Training time comparison across models by sequence length. ParalESN and S5 scale significantly better than the Transformer for sequence lengths exceeding $\approx 8,000$

The Transformer exhibits optimal execution times for contexts under 8,192 tokens. However, beyond this threshold, the $O(L^2)$ complexity of the Self-Attention mechanism imposes a severe temporal penalty. In contrast, ParalESN and S5 capitalize on their linear recurrence and associative scan parallelization to achieve linear scaling, requiring nearly half the processing time of the Transformer at 22,528 tokens.

4.5.4 VRAM Usage

Memory optimization is a critical architectural requirement. We measured VRAM footprint across two axes: expanding context length (holding model dimension at $\approx 30\text{M}$ parameters) and expanding model parameters (holding context length at 1,024).

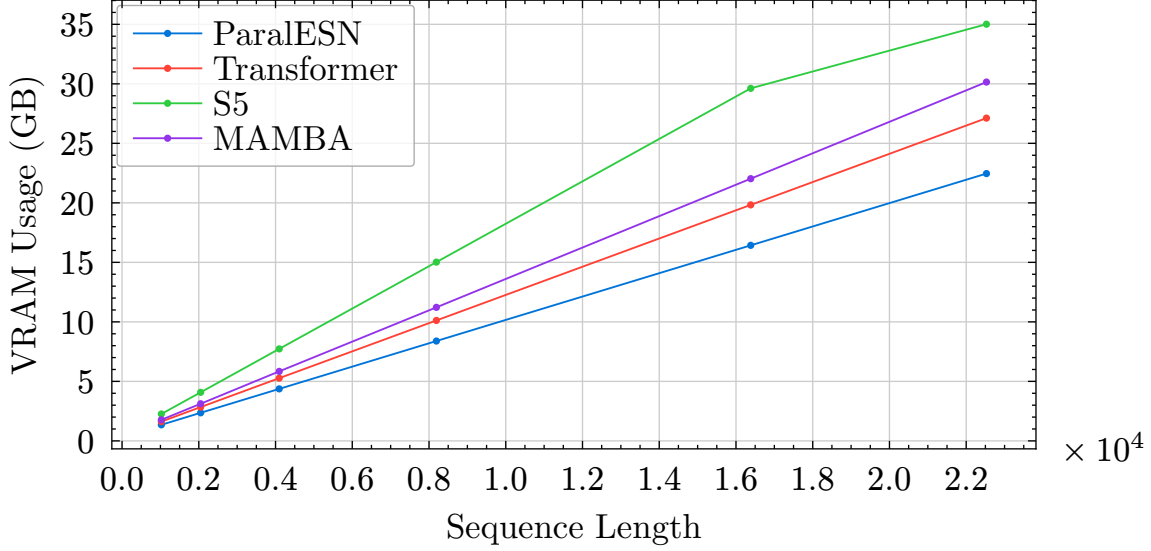


Figure 4.4: VRAM usage comparison across models vs. sequence length. Here, ParaleSN’s VRAM usage scales linearly, unlike the other models.

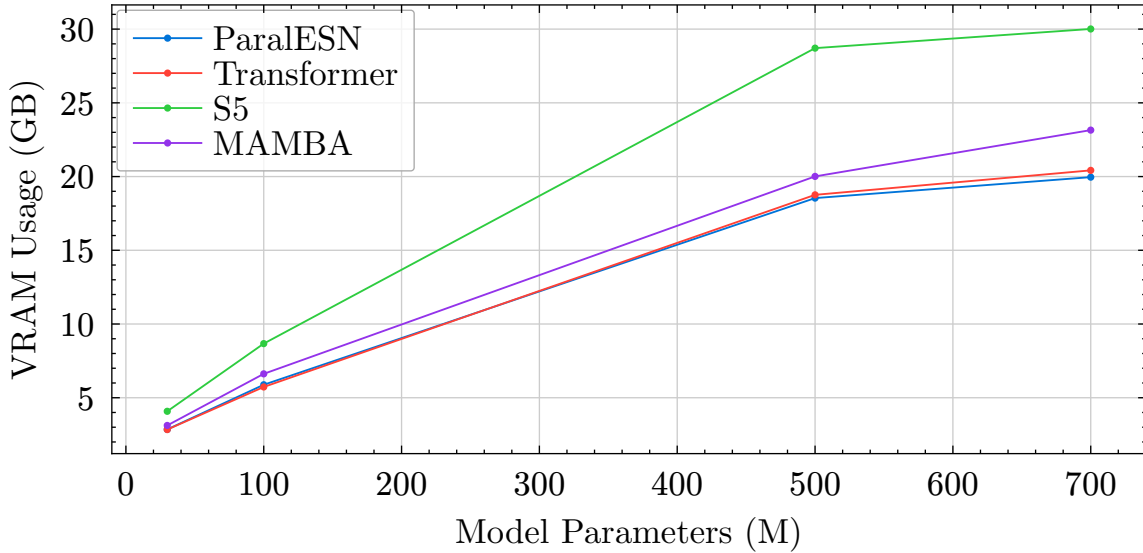


Figure 4.5: VRAM usage comparison across models vs. model dimension. Here, S5 requires more VRAM as model dimension increases. Sequence length is fixed to 1,024.

As demonstrated in the top plot, ParaleSN consistently consumes the least amount of VRAM as the sequence length increases, exhibiting superior memory efficiency compared to all other models. Additionally, the bottom

plot shows that ParalESN maintains a low memory footprint across increasing model dimensions (while keeping the same sequence length of 1,024), closely matching or beating the Transformer architecture. These findings confirm the scalability and efficiency of ParalESN when handling long sequences under limited GPU resources.

4.5.5 Inference Comparison

In the context of Masked Diffusion Language Models, inference time is fundamentally dictated by the time required to denoise a fully masked sequence across iterative diffusion steps. Therefore, the processing latency per step determines the overall throughput.

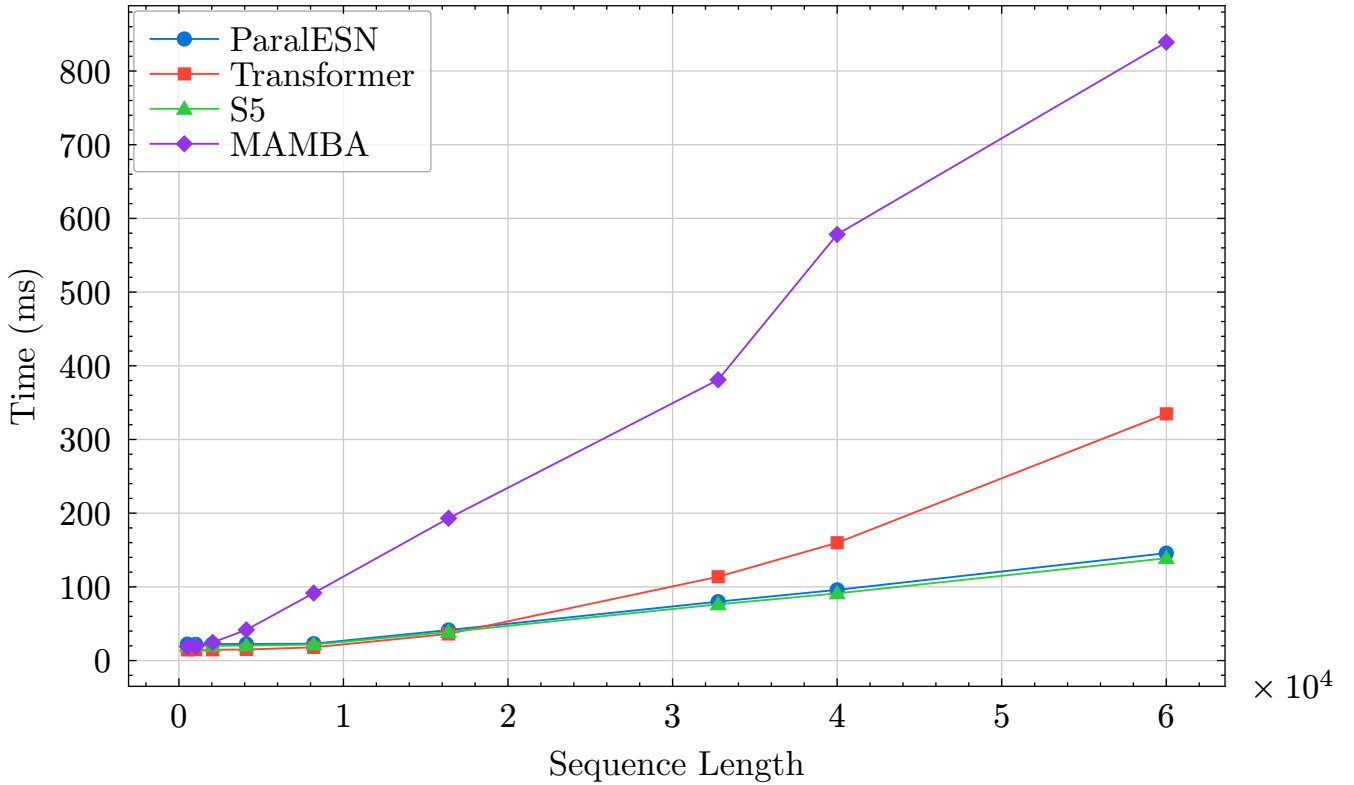


Figure 4.6: Inference time comparison across models. Here, we can clearly see the performance gap between linear models (ParalESN and S5) and the quadratic scaling of the Transformer. MAMBA performs the worst due to the dimension expansion in its SwiGLU [56] layer.

While the Transformer leverages high hardware utilization for short sequences, it suffers substantial deceleration beyond 16,384 tokens. ParalESN and S5 maintain exceptional inference speeds, establishing them as supe-

rior candidates for continuous, real-time generation in Diffusion Language Models processing massive contexts.

4.5.6 DiESN vs. Autoregressive (Qwen1.5-0.5B)

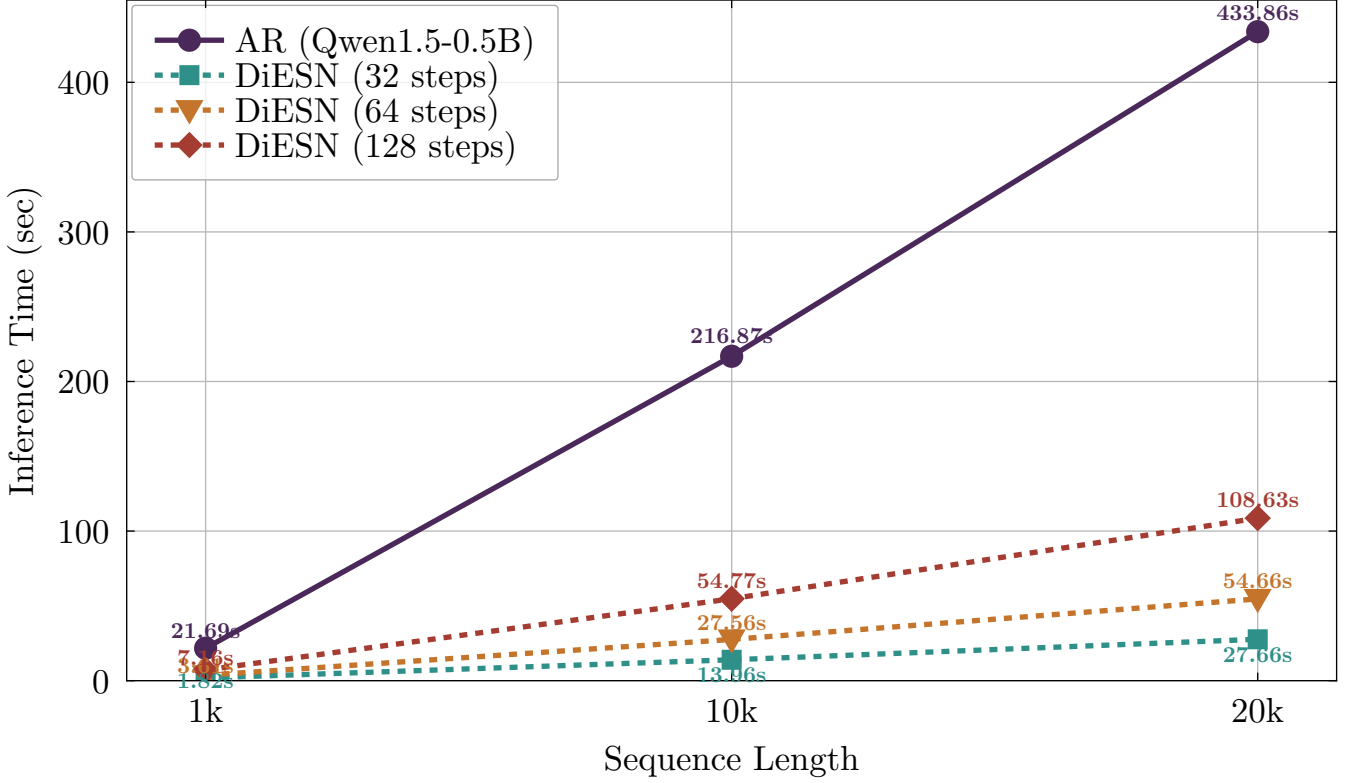


Figure 4.7: Inference time comparison between the autoregressive Qwen1.5-0.5B model [57] and the custom Reservoir-based DiESN diffusion architecture at different sequence lengths. All models have the same effective parameter count (471.8M), while the diffusion model is evaluated with 32, 64, and 128 diffusion steps.

These experimental results highlight a significant runtime advantage of the custom Reservoir-based diffusion architecture (DiESN) over the autoregressive baseline (AR Qwen1.5-0.5B [57]) at a comparable parameter scale ($\approx 471.8\text{M}$ effective parameters). While the autoregressive model scales linearly with sequence length, the diffusion model exhibits a much flatter growth rate. Consequently, DiESN achieves up to a $15.7\times$ speedup at 20k tokens, demonstrating how non-autoregressive generation can bypass the sequential token-by-token bottleneck. Importantly, this evaluation strictly focuses on computational throughput and generation speed; it does not assess output quality or task performance, which would require dedicated benchmark comparisons.

4.5.7 Ablation

In this experiment, we assess the general contribution of ParalESN. Specifically, we evaluate three distinct ablations:

- In relation to Figure 3.1 and Figure 3.2, the ParalESN branch has been removed to retain only the convolutional layer and the GLA during training on the experiment of Section 4.3.1
- Similar as before but replacing the ParalESN branch with a simple linear layer (DiESN w. Linear) while keeping all the rest, on the same TinyStories experiment
- Making both the ParalESN recurrence matrix and the mixer fully trainable and evaluating the model on the first 5,000 steps on OpenWebText (same experiment of Section 4.4.2).

Model	PPL	Time	Train. Params	Fixed Params	Total Params
DiESN	11.82	1:22:09	20.1 M	3.9 M	24.0 M
DiESN (w/o ParalESN)	340.36	1:13:09	20.1 M	–	20.1 M
DiESN (w. Linear)	<u>80</u>	<u>1:20:04</u>	24.0 M	–	24.0 M

Table 4.15: Comparison on TinyStories between the full DiESN model, the model without the ParalESN branch, and the model where ParalESN is replaced by a linear layer.

Model	Time	Train. Params	Fixed Params	Total Params
S5	1:15:39	615 M	–	615 M
ParalESN	0:51:31	481 M	168 M	649 M
ParalESN (trained)	<u>1:08:34</u>	649 M	–	649 M

Table 4.16: Performance comparison between S5 and ParalESN models on the first 5,000 steps of OpenWebText. The test highlights how the non-trainability of ParalESN leads to a reduction of $\approx 24\%$.

The results in Table 4.15 highlight the critical role of the ParalESN branch. Removing ParalESN forces the network to rely solely on the depthwise convolutional layer and the GLA, resulting in a severe performance collapse and a massive increase in perplexity. This confirms that local convolution and GLA alone are insufficient for effective sequence modeling. Replacing

ParalESN with a linear layer only partially recovers the performance: the perplexity drops to 80, but it remains far from the 11.82 of the full model, even though both models have the same total number of parameters (24.0 M) and the linear variant trains all of them.

Also, as shown in Table 4.16, enabling parameter trainability in the ParalESN framework introduces a noticeable computational overhead. In smaller architectures, the difference in training time between the fixed and the fully trained variants is negligible. However, as the model grows and the share of fixed parameters increases, keeping the reservoir frozen becomes a clear advantage: in the largest configuration, ParalESN trains in about 51 minutes instead of 1h 08m, saving roughly a quarter of the training time.

4.6 ParalESN vs. S5: The Impact of Accelerated Scan

Although S5 often delivers performance (in terms of PPL) comparable to ParalESN, its architecture exhibits scalability limitations. The unofficial S5 implementation employs a Triton-based [58] kernel for associative scanning, prioritizing generalizability over pure GPU optimization; in contrast, ParalESN avoids this drawback by leveraging highly accelerated, native-CUDA [59] prefix-sum logic. To pinpoint the actual differences, the S5 library was modified to use the same scanning routine as ParalESN, followed by the training of models of various sizes. Since the specific objective was to compare training speed, the first 5,000 steps of OpenWebText dataset were used.

Parameter	Value
Context Length	1024
Hidden Size	512
N. of layers	64
Batch Size	128
Learning Rate	3×10^{-4}
N. of steps	5,000

Table 4.17: Training Specifications for Architectural Scalability Test

Model	Trainable Parameters	Fixed Parameters	Total Parameters
ParalESN	29.1M	5.2M	34.3M
S5	33.3M	–	33.3M
ParalESN	44.9M	10M	54.9M
S5	53.3M	–	53.3M
ParalESN	76.6M	21M	97.6M
S5	93.4M	–	93.4M
ParalESN	83.8M	21M	104.8M
S5	100M	–	100M
ParalESN	139M	42M	181M
S5	173M	–	173M
ParalESN	140M	42M	182M
S5	174M	–	174M
ParalESN	254M	84M	338M
S5	321M	–	321M
ParalESN	481M	168M	649M
S5	615M	–	615M

Table 4.18: Models used for the comparison between S5 and ParalESN in terms of parameters.

In this context, we therefore want to see:

- How model behavior changes as the parameter count increases, and whether non-trainable parameters play a significant role (Figure 4.8),
- How much execution time can be saved across an increasing number of training steps (Figure 4.9).

In Figure 4.8, all execution times were measured, whereas in Figure 4.9, only the first four execution times were measured; the others are estimated.

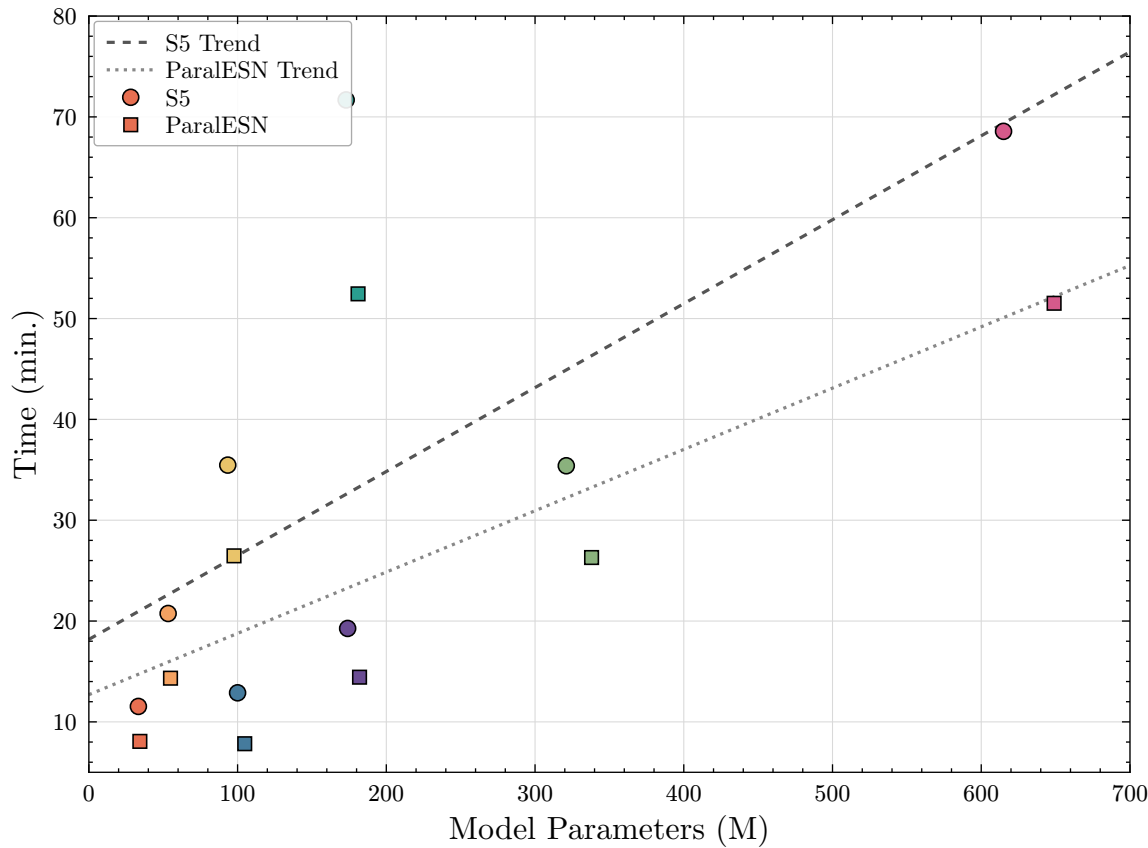


Figure 4.8: Comparison of execution times vs. model parameters. As expected, as the number of trainable parameters grows, ParaleSN has a larger percentage of fixed parameters, and this permits linear scaling.

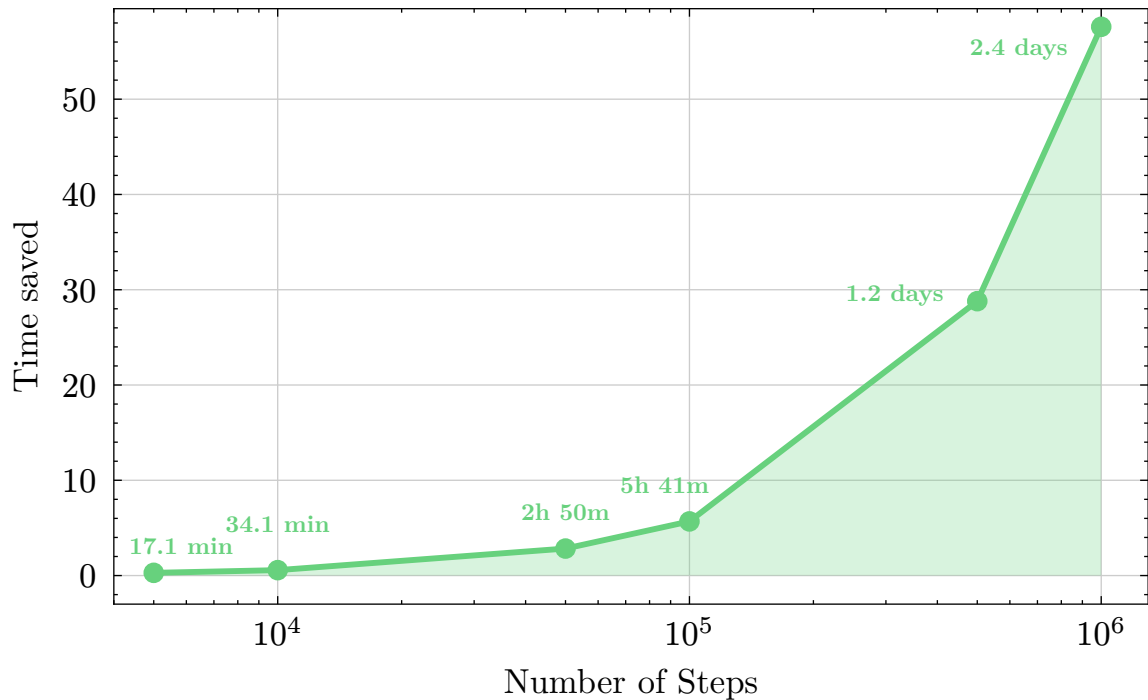


Figure 4.9: Time savings across steps: the first four runtimes are measured, while the remaining ones are estimated.

As illustrated in Figure 4.8 and Figure 4.9, increasing the backbone parameters exposes critical bottlenecks. When hitting the 300M+ parameters, S5 temporal cost scales super-linearly. Conversely, ParalESN follows a linear temporal trajectory. For a 600M parameter model, using ParalESN instead of S5 saves several days of computation.

These results clearly show the superiority of ParalESN over S5 in terms of training efficiency. ParalESN is faster in every configuration we tested, reducing the training time by 25% to 39%, even though in each pair it has slightly more total parameters than S5. For the largest model (649M parameters), ParalESN completes the 5000 training steps in 51 minutes, while S5 needs almost 69 minutes. The trend lines of Figure 4.8 also show that the training time of ParalESN grows about 27% more slowly with model size than that of S5, so the gap keeps widening as models become larger: over one million training steps, the estimated saving reaches about 2.4 days (Figure 4.9).

It is important to stress that, in this experiment, both models use exactly the same scan routine. The advantage of ParalESN is therefore not due to a better implementation, but to its architecture: since the reservoir parameters are fixed, no gradients have to be computed for them and no optimizer states have to be stored, which saves both computation and memory at every training step.

Overall, these results confirm that ParalESN is not only a highly scalable alternative for integrating linear recurrence into diffusion-based language generation frameworks, but also the most efficient choice between the two linear recurrent backbones.

Chapter 5

Conclusions

In this thesis, we have explored the integration of Reservoir Computing (RC) into the modern landscape of generative language modeling. Specifically, we addressed the computational bottlenecks of traditional Transformer-based architectures in handling long sequences. To overcome the quadratic complexity of self-attention, which often necessitates sequence chunking, we proposed a novel diffusion-based language model where the conventional Transformer backbone is replaced by ParalESN, an efficient, untrained linear recurrent layer.

Our primary objective was to adapt a soft-masked diffusion model for text generation, leveraging the highly parallelizable diagonal linear recurrence of ParalESN. Initially, a key motivation for this architectural shift was the prospect of drastically reducing the total number of trainable parameters by relying on fixed reservoir dynamics. However, empirical development revealed that a significant portion of the trainable parameter budget resides within the Multi-Layer Perceptrons (MLPs) of each residual block, which remain essential for non-linear feature transformation and overall model expressivity.

Despite this realization regarding parameter distribution, the proposed ParalESN-driven diffusion model demonstrated substantial computational advantages. Our results indicate that ParalESN successfully mitigates the time complexity issues associated with long contexts, outperforming attention-based baselines in training speed for extended sequences. Furthermore, the model maintained competitive Perplexity (PPL) values across both short and long sequences, validating the representational capacity of the untrained reservoir and establishing it as a viable and highly efficient alternative to classical Transformers in diffusion frameworks.

Beyond training efficiency, we extensively evaluated the generative capabilities of the model during inference, demonstrating faster generation speed compared to other models.

Looking forward, this work opens several promising avenues for future research:

- **Mixture of Experts (MoE) Integration:** A natural extension would be replacing the standard MLPs with a Mixture of Experts architecture [60]. These experts could be left untrained or partially trained, potentially integrating ParalESN itself as a specialized routing expert to further optimize computational resources while maintaining expressivity.
- **Large-Scale Pretraining:** Scaling up the proposed architecture to train on massive datasets would allow for a comprehensive benchmark against state-of-the-art frontier models, thoroughly assessing the limits of diffusion-based reservoir computing in natural language processing.
- **Hyperparameter Optimization and Freezing:** The extensive hyperparameter space of ParalESN poses a challenge at scale. A hybrid approach could involve a brief initial training phase to dynamically learn and stabilize the reservoir’s optimal parameters (e.g., spectral radius and phases) via gradient descent, subsequently freezing them to preserve the Echo State Property (ESP) and retain the benefits of untrained dynamics.
- **Autoregressive Adaptation:** Finally, evaluating ParalESN within a standard autoregressive language modeling paradigm would provide valuable insights into its relative strengths and weaknesses compared to its performance in the continuous, parallel refinement process of soft-masked diffusion.

Ultimately, by demonstrating the efficacy of ParalESN within a diffusion-based generative framework, this thesis contributes to the ongoing efforts to make sequence modeling more efficient, scalable, and accessible, reaffirming the relevance of reservoir computing principles in the modern deep learning era.

The core contributions of this thesis have been accepted for presentation at the NeurIPS 2026 Workshop (AXIOM) [15].

Bibliography

- [1] A. Vaswani *et al.*, “Attention Is All You Need.” [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [2] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models.” [Online]. Available: <https://arxiv.org/abs/2006.11239>
- [3] X. L. Li, J. Thickstun, I. Gulrajani, P. Liang, and T. B. Hashimoto, “Diffusion-LM Improves Controllable Text Generation.” [Online]. Available: <https://arxiv.org/abs/2205.14217>
- [4] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured Denoising Diffusion Models in Discrete State-Spaces.” [Online]. Available: <https://arxiv.org/abs/2107.03006>
- [5] M. Hersche, S. Moor-Smith, T. Hofmann, and A. Rahimi, “Soft-Masked Diffusion Language Models.” [Online]. Available: <https://arxiv.org/abs/2510.17206>
- [6] J. L. Elman, “Finding Structure in Time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990, doi: 10.1207/s15516709cog1402_1.
- [7] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994, doi: 10.1109/72.279181.
- [8] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *International Conference on Machine Learning (ICML)*, 2013, pp. 1310–1318. [Online]. Available: <https://arxiv.org/abs/1211.5063>
- [9] P. J. Werbos, “Backpropagation through time: what it does and how to do it,” *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990, doi: 10.1109/5.58337.
- [10] D. Verstraeten, B. Schrauwen, M. D’Haene, and D. Stroobandt, “An experimental unification of reservoir computing methods,” *Neural Networks*, vol. 20, no. 3, pp. 391–403, 2007, doi: 10.1016/j.neunet.2007.04.003.

- [11] M. Lukoševičius and H. Jaeger, “Reservoir computing approaches to recurrent neural network training,” *Computer Science Review*, vol. 3, no. 3, pp. 127–149, 2009, doi: 10.1016/j.cosrev.2009.07.001.
- [12] H. Jaeger, “The "echo state" approach to analysing and training recurrent neural networks-with an erratum,” technical report 148, 2001.
- [13] M. Pinna, G. Lagomarsini, A. Ceni, and C. Gallicchio, “ParalESN: Enabling parallel information processing in Reservoir Computing.” [Online]. Available: <https://arxiv.org/abs/2601.22296>
- [14] G. E. Blelloch, “Prefix Sums and Their Applications,” in *Synthesis of Parallel Algorithms*, J. H. Reif, Ed., Morgan Kaufmann, 1990.
- [15] G. Benedetti, G. Lagomarsini, A. Ceni, and C. Gallicchio, “Towards Efficient Diffusion Language Modeling via Parallel Reservoir Computing,” in *NeurIPS 2026 Workshop AXIOM*, 2026. [Online]. Available: <https://openreview.net/forum?id=5aVnVHCPqV>
- [16] K. Cho *et al.*, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734. [Online]. Available: <https://arxiv.org/abs/1406.1078>
- [17] F. Contributors, “Flash Linear Attention: Efficient Hardware-Aware Designs for Recurrent Neural Networks.” 2024.
- [18] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim, “Gated Linear Attention Transformers with Hardware-Efficient Training.” [Online]. Available: <https://arxiv.org/abs/2312.06635>
- [19] S. Arora *et al.*, “Zoology: Measuring and Improving Recall in Efficient Language Models,” *arXiv preprint arXiv:2312.04927*, 2023.
- [20] A. Gu *et al.*, “Combining Recurrent, Convolutional, and Continuous-time Models with Linear State Space Layers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 572–585. [Online]. Available: <https://arxiv.org/abs/2110.13985>
- [21] A. Gu, K. Goel, and C. Ré, “Efficiently Modeling Long Sequences with Structured State Spaces.” [Online]. Available: <https://arxiv.org/abs/2111.00396>

- [22] A. Gu, T. Dao, S. Ermon, A. Rudra, and C. Ré, “HiPPO: Recurrent Memory with Optimal Polynomial Projections,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1474–1487. [Online]. Available: <https://arxiv.org/abs/2008.07669>
- [23] J. T. Smith, A. Warrington, and S. W. Linderman, “Simplified State Space Layers for Sequence Modeling,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://arxiv.org/abs/2212.07632>
- [24] A. Gu, K. Goel, A. Gupta, and C. Ré, “On the Parameterization and Initialization of Diagonal State Space Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 35971–35983. [Online]. Available: <https://arxiv.org/abs/2206.11893>
- [25] A. Orvieto *et al.*, “Resurrecting Recurrent Neural Networks for Long Sequences,” in *International Conference on Machine Learning (ICML)*, 2023, pp. 26670–26698. [Online]. Available: <https://arxiv.org/abs/2303.06349>
- [26] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces.” [Online]. Available: <https://arxiv.org/abs/2312.00752>
- [27] T. Dao and A. Gu, “Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality.” [Online]. Available: <https://arxiv.org/abs/2405.21060>
- [28] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli, “Deep Unsupervised Learning using Nonequilibrium Thermodynamics,” in *International Conference on Machine Learning (ICML)*, 2015, pp. 2256–2265. [Online]. Available: <https://arxiv.org/abs/1503.03585>
- [29] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 6840–6851. [Online]. Available: <https://arxiv.org/abs/2006.11239>
- [30] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*,

- 2019, pp. 4171–4186. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [31] X. L. Li, J. Thickstun, I. Gulrajani, P. Liang, and T. B. Hashimoto, “Diffusion-LM Improves Controllable Text Generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 4328–4343. [Online]. Available: <https://arxiv.org/abs/2205.14217>
- [32] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and R. van den Berg, “Structured Denoising Diffusion Models in Discrete State-Spaces,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 17981–17993. [Online]. Available: <https://arxiv.org/abs/2107.03006>
- [33] S. S. Sahoo, A. Gokaslan, Z. Wu, D. Y. Chiu, and J. You, “Simple and Effective Masked Diffusion Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, pp. 116480–116513. [Online]. Available: <https://arxiv.org/abs/2406.07524>
- [34] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, “Score-Based Generative Modeling through Stochastic Differential Equations,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://arxiv.org/abs/2011.13456>
- [35] S. Dieleman *et al.*, “Continuous Diffusion for Categorical Data.” [Online]. Available: <https://arxiv.org/abs/2211.15089>
- [36] I. Gulrajani and T. B. Hashimoto, “Likelihood-Based Diffusion Language Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.18619>
- [37] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 4195–4205. [Online]. Available: <https://arxiv.org/abs/2212.09748>
- [38] K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1026–1034. [Online]. Available: <https://arxiv.org/abs/1502.01852>

- [39] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs).” [Online]. Available: <https://arxiv.org/abs/1606.08415>
- [40] J. Snoek, H. Larochelle, and R. P. Adams, “Practical Bayesian Optimization of Machine Learning Algorithms,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 2951–2959.
- [41] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?” [Online]. Available: <https://arxiv.org/abs/2305.07759>
- [42] R. Eldan and Y. Li, “TinyStories Dataset on Hugging Face.” 2023.
- [43] S. Gunasekar *et al.*, “Textbooks Are All You Need.” [Online]. Available: <https://arxiv.org/abs/2306.11644>
- [44] SciPhi, “Textbooks Are All You Need (Lite) Dataset on Hugging Face.” 2023.
- [45] wikimedia, “Wikipedia Dataset on Hugging Face.” 2023.
- [46] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “WikiText-103 Dataset on Hugging Face.” 2016.
- [47] A. Gokaslan and V. Cohen, “OpenWebText Corpus.” 2019.
- [48] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners.” 2019.
- [49] A. Gokaslan and V. Cohen, “OpenWebText Dataset on Hugging Face.” 2019.
- [50] S. Jelassi, D. Brandfonbrener, S. M. Kakade, and E. Malach, “Repeat After Me: Transformers are Better than State Space Models at Copying.” [Online]. Available: <https://arxiv.org/abs/2402.01032>
- [51] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024, [Online]. Available: <https://arxiv.org/abs/2104.09864>
- [52] O. Press, N. A. Smith, and M. Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: <https://arxiv.org/abs/2108.12409>

- [53] A. Kazemnejad, I. Padhi, K. N. Ramamurthy, P. Das, and S. Reddy, “The Impact of Positional Encoding on Length Generalization in Transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.19466>
- [54] S. Chen, S. Wong, L. Chen, and Y. Tian, “Extending Context Window of Large Language Models via Positional Interpolation.” [Online]. Available: <https://arxiv.org/abs/2306.15595>
- [55] M. Suzgun, S. Gehrmann, Y. Belinkov, and S. M. Shieber, “Evaluating the Memory Capacity of Neural Sequence Models with Dyck Languages,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 1205–1213. [Online]. Available: <https://arxiv.org/abs/1905.04944>
- [56] N. Shazeer, “GLU Variants Improve Transformer.” [Online]. Available: <https://arxiv.org/abs/2002.05202>
- [57] Qwen Team, “Qwen1.5: A Step Towards Stronger Foundation Models.” 2024.
- [58] P. Tillet, H. T. Kung, and D. Cox, “Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations,” in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19. doi: 10.1145/3315508.3329973.
- [59] J. Nickolls, I. Buck, M. Garland, and K. Skadron, “Scalable Parallel Programming with CUDA,” *ACM Queue*, vol. 6, no. 2, pp. 40–53, 2008, doi: 10.1145/1365490.1365500.
- [60] N. Shazeer *et al.*, “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1701.06538>